Day 9 Hands-on Assignment

Optimizing and Recovering an Oracle Database

John Michael Bilbao
Techstart
August 28, 2026

What this covers

This assignment builds an Oracle database, measures a query against it, indexes the query, then destroys a table and brings it back from an RMAN backup. Everything in it ran against a real instance: ----------, started from the gvenzl/oracle-free container image on eight cores with a 1.5 GB SGA.

Every figure is output captured from that instance. The capture files live in results/, scripts/make_figures.py renders those files into the figures below, and none of it is retyped by hand, so no figure can show something the database did not do. The numbers quoted in the text are parsed from the same files.

> The short version of what was found: the index the assignment asks for does not speed up the query the assignment asks for, and the execution plans show exactly why. It speeds up a neighbouring query by 10.7x, and a two-column version of it makes the original query read 43 percent fewer blocks while still taking longer on the clock. All three results are below with the plans and the timings that produced them.

Introduction: creating the database

The instance runs in a container, which is the closest thing to a fresh install that can be repeated exactly. scripts/start_db.sh creates it and waits for it to open; the container's own startup narration is the record of the creation.

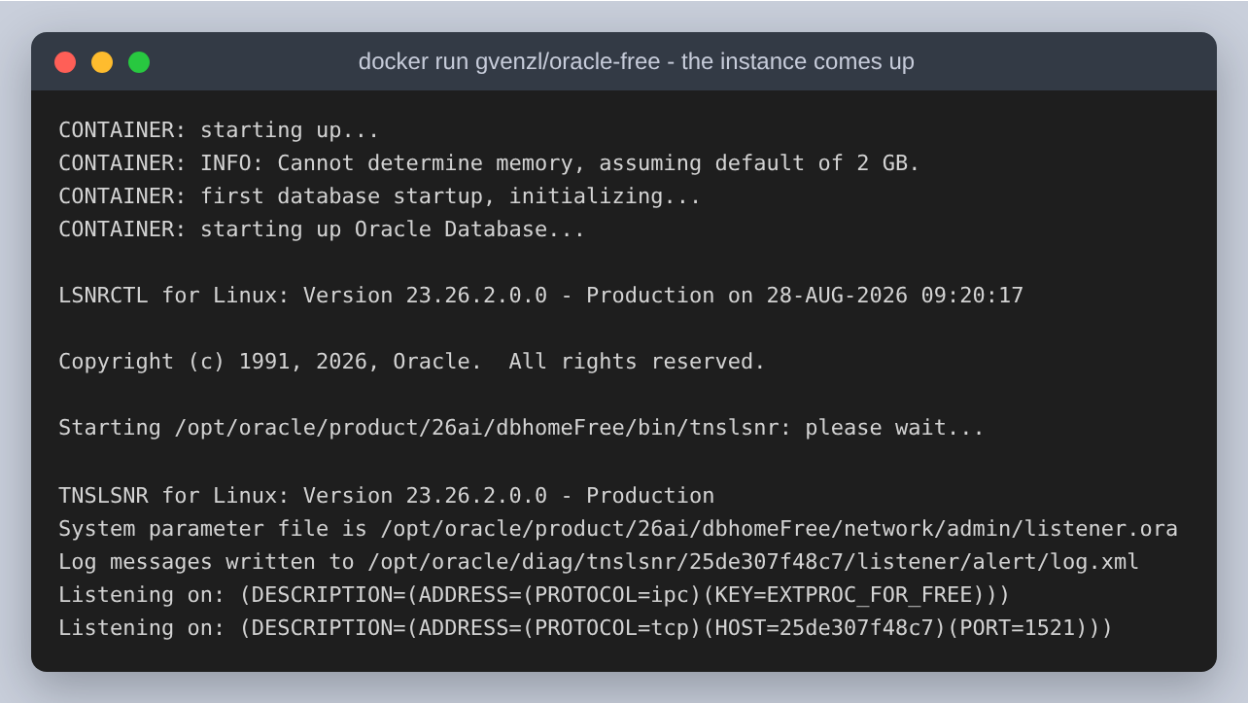

Figure 1: The instance being created: the container initialises the database and starts the TNS listener on port 1521.

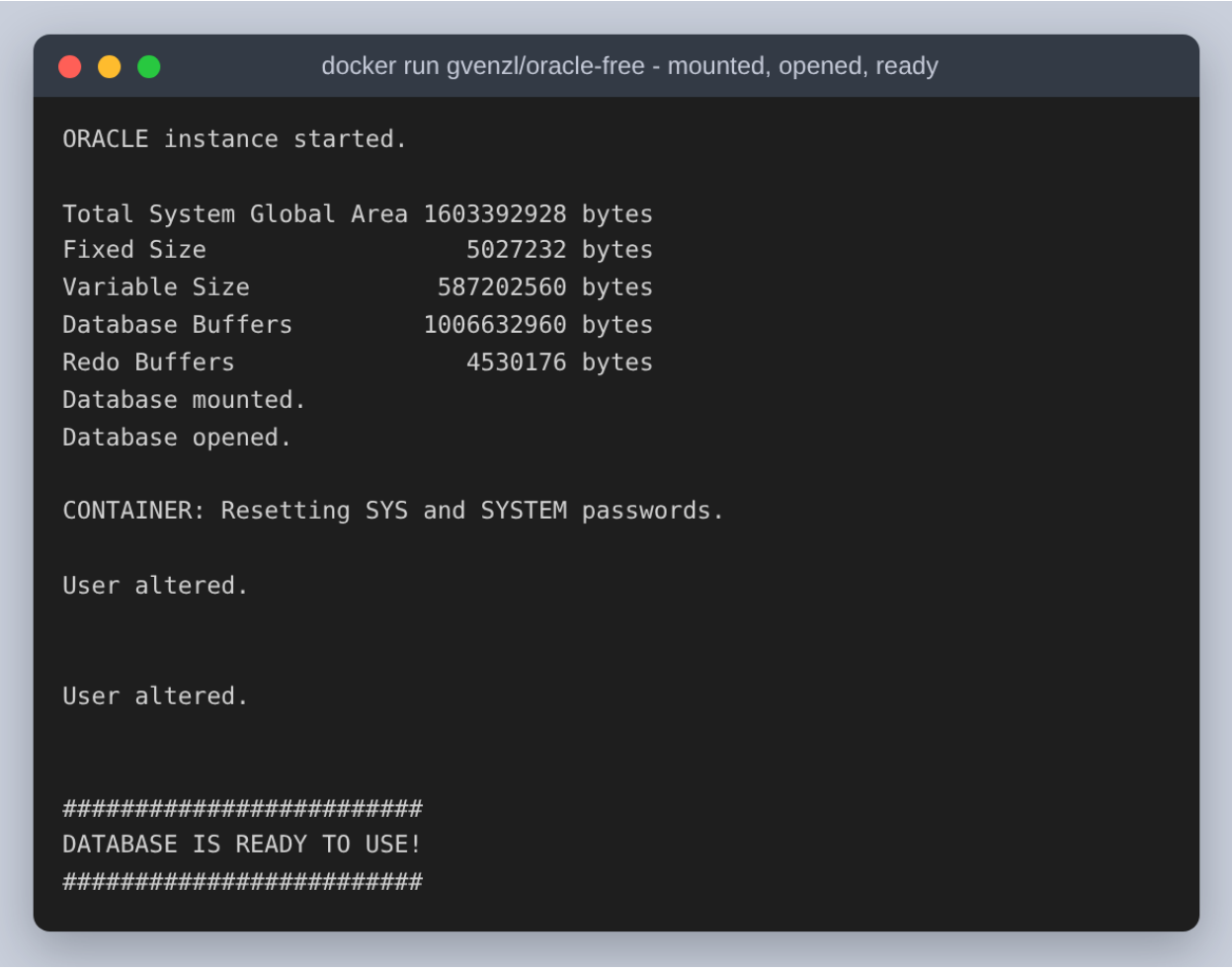

Figure 2: The same startup finishing: the SGA is allocated, the database is mounted and opened, and the container reports it ready to use.

Oracle 23ai is multitenant, so what came up is not one database but a container database, FREE, holding a seed and one pluggable database, FREEPDB1. That distinction matters twice later on: the schema is built inside FREEPDB1, while ARCHIVELOG mode and the RMAN backup are properties of the container database around it.

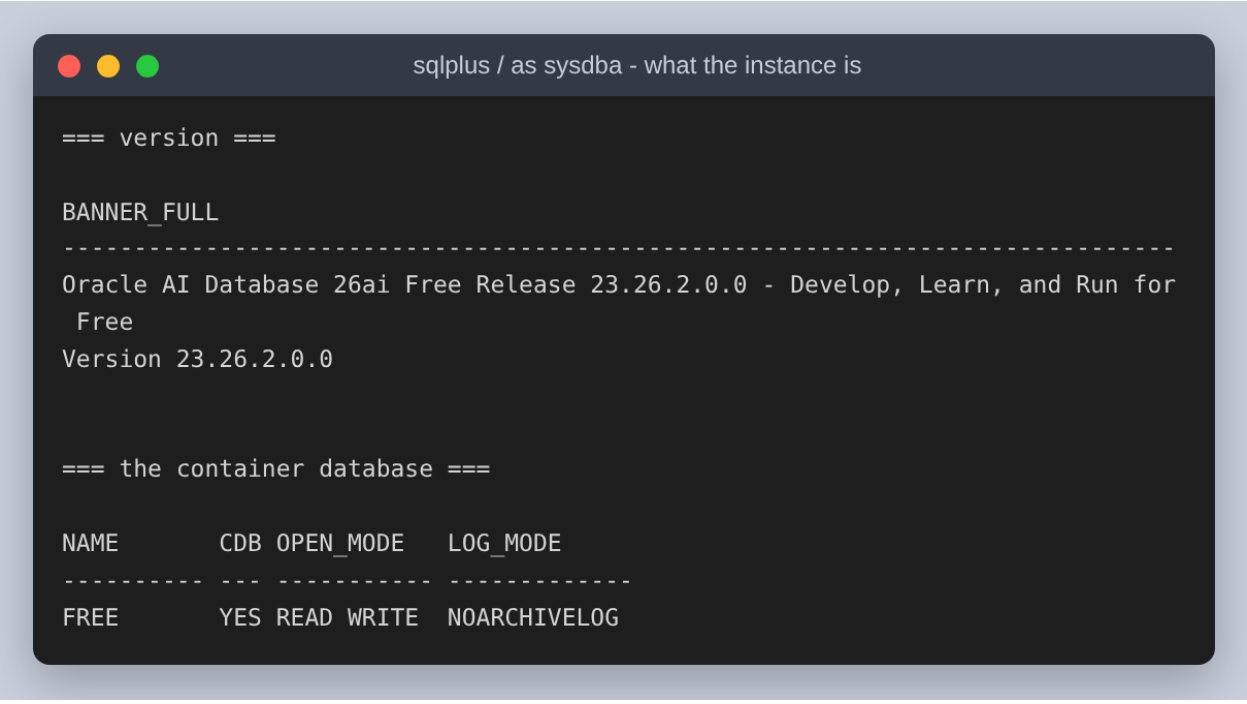

Figure 3: The version and the container database. Note LOG_MODE: the image ships in NOARCHIVELOG, which Step 3 has to change before it can back anything up usefully.

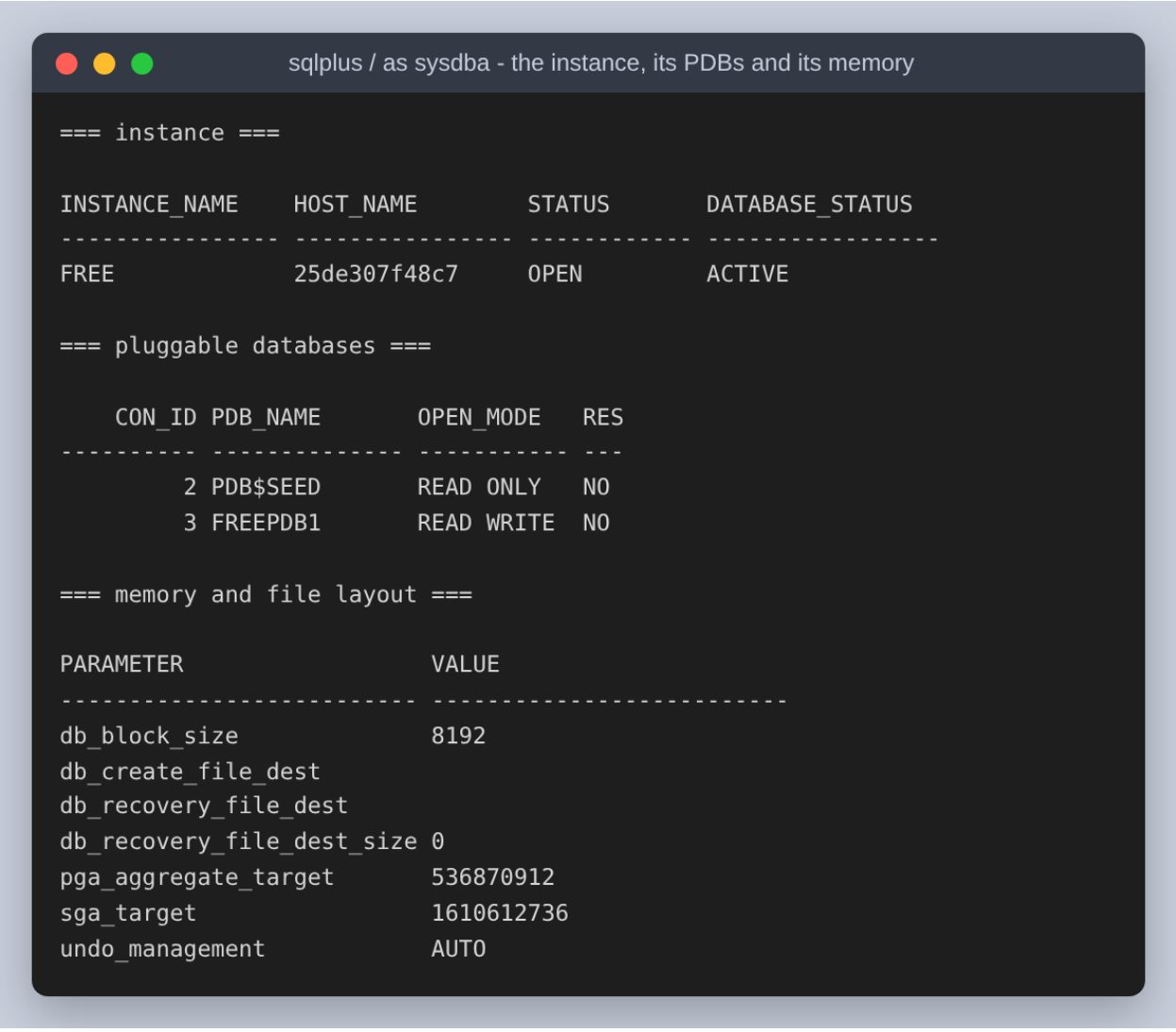

Figure 4: The instance, the two pluggable databases inside it, and the memory and file settings the rest of the assignment runs under.

Step 1: creating the tables and the index

departments is created first, because employees.department_id references it and a foreign key cannot point at a table that does not exist yet. Both tables get a primary key, and employees gets a check constraint on salary as well, so that the recovery in Step 3 has something to prove came back besides the rows.

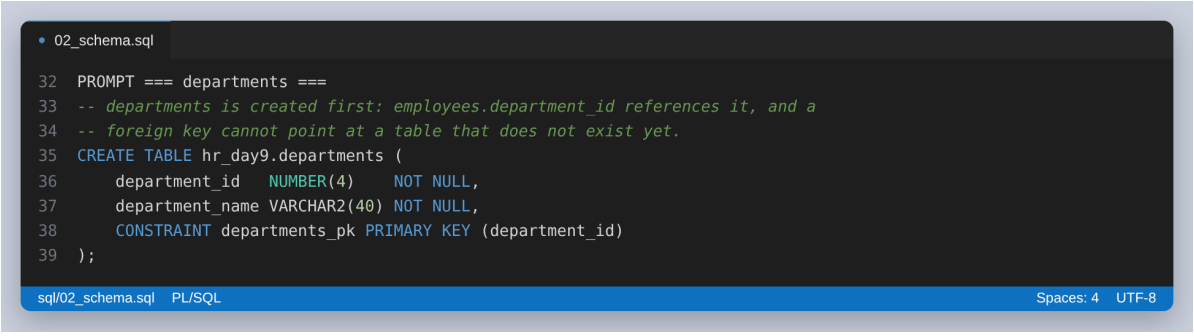

Figure 5: departments: the parent table, created first.

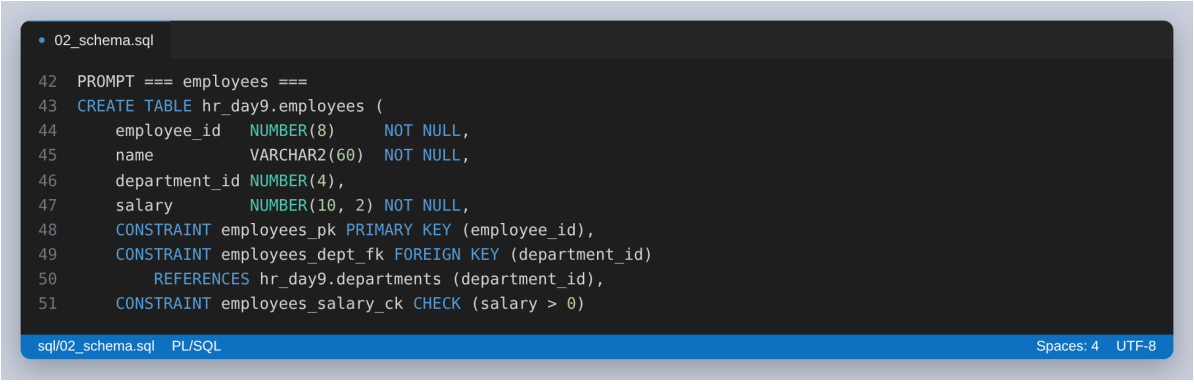

Figure 6: employees: primary key, the foreign key to departments, and a check constraint that a salary has to be positive.

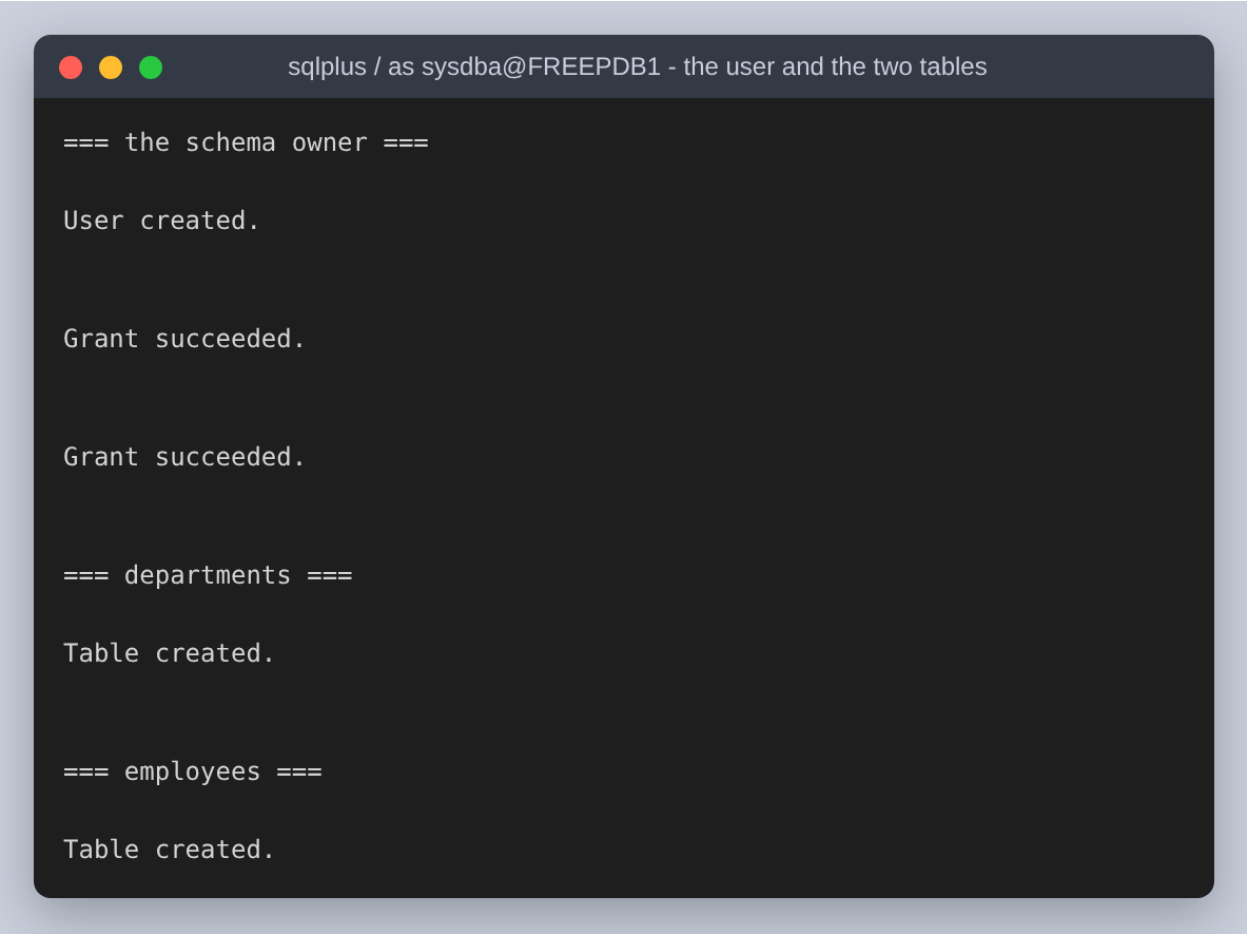

Figure 7: The schema owner and both tables being created. HR_DAY9 is an ordinary application user; SELECT_CATALOG_ROLE is granted only so that it can read execution plans out of v$sql_plan in Step 2.

Ten departments and twelve employees are inserted, which is two more employees than the assignment asks for and lets two departments hold more than one person, so the averages in Step 2 are averages of something rather than of a single row.

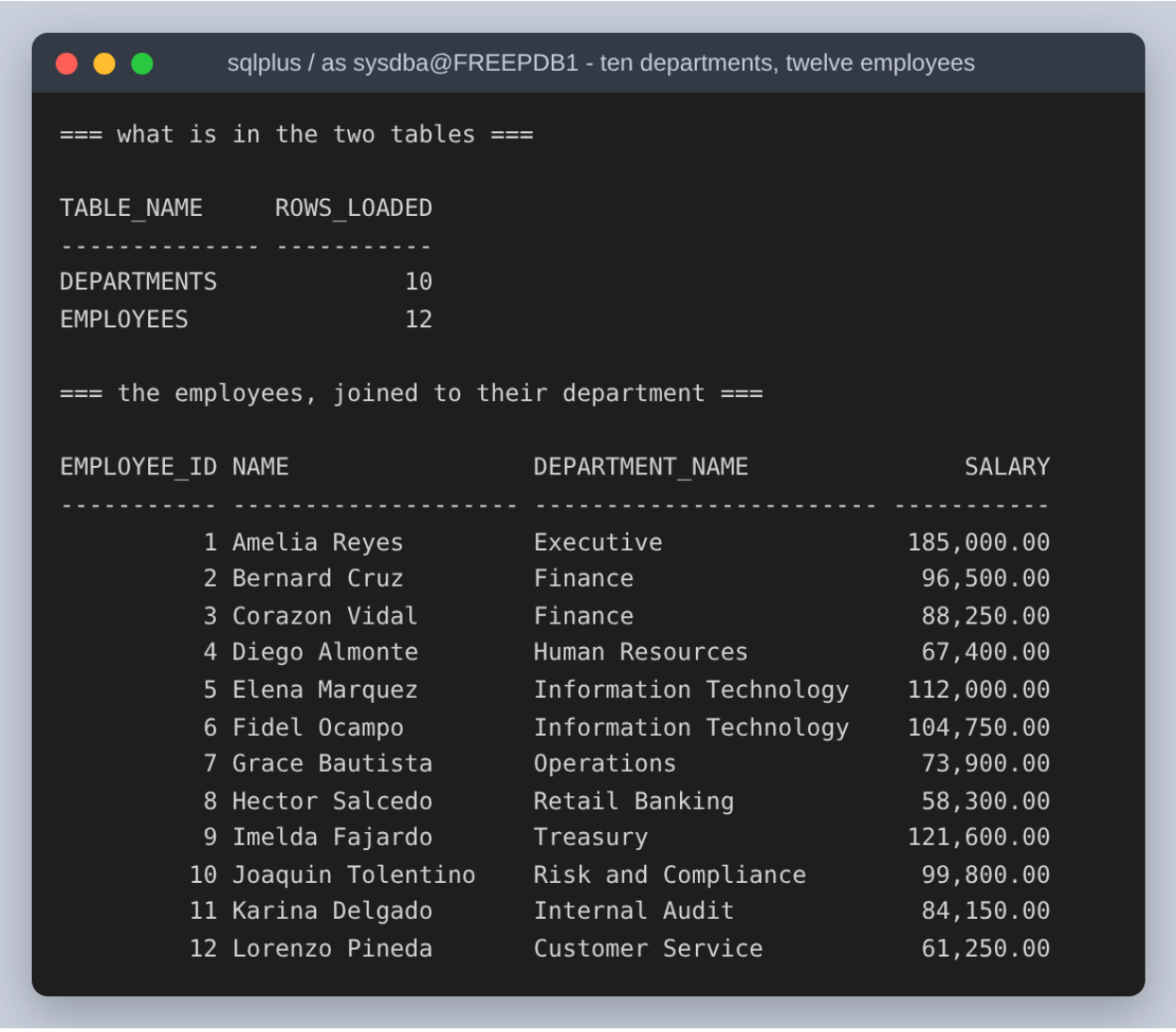

Figure 8: The row counts after the inserts, and every employee joined to the department that the foreign key points at.

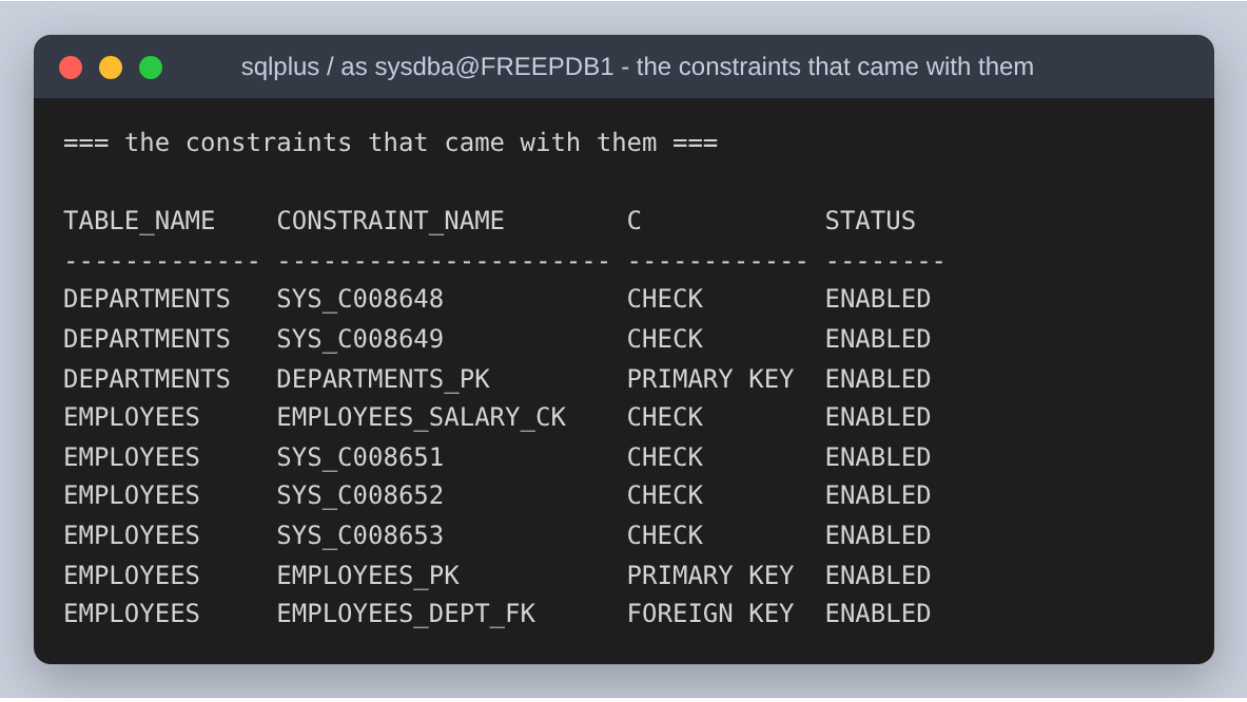

Figure 9: The constraints that came with the two tables. The SYS_C-prefixed checks are the NOT NULL declarations, which Oracle stores as check constraints with generated names.

The index the assignment asks for is on employees.department_id. It is worth being clear that it is the third index in the schema and not the first: Oracle built a unique index behind each primary key automatically, and those exist whether they are wanted or not. This one is the only index so far that is a deliberate choice, and therefore the only one whose value is worth arguing about.

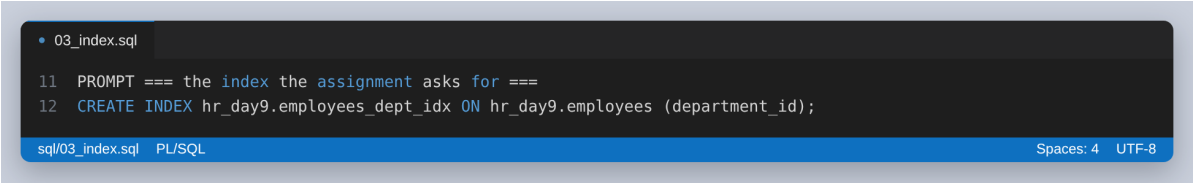

Figure 10: The index on employees.department_id.

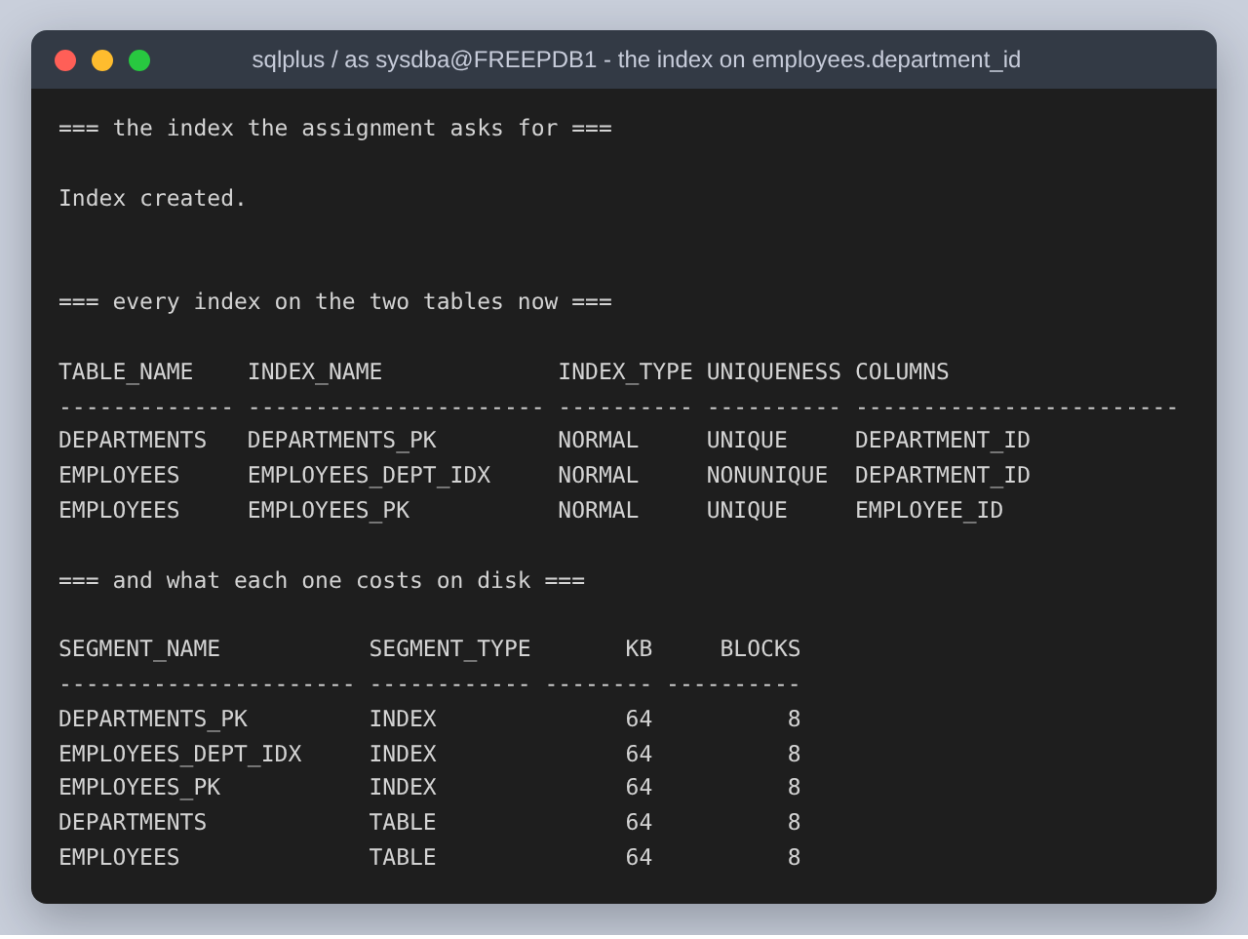

Figure 11: The index being created, all three indexes on the two tables, and what each costs on disk. At twelve rows every segment is one 64 KB extent, so the index costs as much space as the table it indexes.

Step 2: the PL/SQL query

The assignment asks for a PL/SQL query returning the average salary of employees in each department. It is written twice: once as an anonymous block that reports every department, and once as a stored function that SQL can call per department.

One decision inside the block is worth stating, because it is the difference between PL/SQL that helps and PL/SQL that hurts. The average is computed by SQL, in the cursor, and the PL/SQL only walks the result and formats it. Fetching all the employee rows into PL/SQL and averaging them in a loop would produce the same answer while moving every row across the call interface to compute a number the database can produce itself.

04_plsql.sql

```
12  DECLARE
13      -- The aggregate is left in SQL and the cursor walks the result. Averaging
14      -- in PL/SQL instead would mean fetching every employee row across the
15      -- call interface to compute a number the database can produce itself.
16      CURSOR c_dept_avg IS
17          SELECT   d.department_id,
18                   d.department_name,
19                   COUNT(e.employee_id)   AS headcount,
20                   AVG(e.salary)          AS avg_salary
21          FROM     hr_day9.departments d
22                   LEFT JOIN hr_day9.employees e
23                          ON e.department_id = d.department_id
24          GROUP BY d.department_id, d.department_name
25          ORDER BY d.department_id;
26
27      l_reported PLS_INTEGER := 0;
28  BEGIN
29      DBMS_OUTPUT.PUT_LINE(RPAD('DEPT', 6) || RPAD('DEPARTMENT', 24)
30                           || LPAD('STAFF', 6) || LPAD('AVG SALARY', 14));
31      DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
32
33      FOR r IN c_dept_avg LOOP
34          -- A department with no employees averages NULL rather than zero, and
35          -- saying so is more honest than printing a 0.00 nobody earns.
36          DBMS_OUTPUT.PUT_LINE(
37              RPAD(r.department_id, 6)
38              || RPAD(r.department_name, 24)
39              || LPAD(r.headcount, 6)
40              || LPAD(NVL(TO_CHAR(r.avg_salary, '9,999,999.99'), '     no staff'), 14));
41          l_reported := l_reported + 1;
42      END LOOP;
43
44      DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
45      DBMS_OUTPUT.PUT_LINE(l_reported || ' departments reported');
46  END;
```

sql/04_plsql.sql PL/SQL Spaces: 4 UTF-8

Figure 12: The anonymous block. The aggregate is left in SQL; the LEFT JOIN means a department with no employees is still reported.

sqlplus / as sysdba@FREEPDB1 - the anonymous block's output

```
=== an anonymous PL/SQL block, one row per department ===
DEPT  DEPARTMENT               STAFF    AVG SALARY
------------------------------------------------
10    Executive                    1    185,000.00
20    Finance                      2     92,375.00
30    Human Resources              1     67,400.00
40    Information Technology       2    108,375.00
50    Operations                   1     73,900.00
60    Retail Banking               1     58,300.00
70    Treasury                     1    121,600.00
80    Risk and Compliance          1     99,800.00
90    Internal Audit               1     84,150.00
100   Customer Service             1     61,250.00
------------------------------------------------
10 departments reported
```

Figure 13: The block's output. All ten departments appear, including the eight that hold exactly one person.

04_plsql.sql

```
52  CREATE OR REPLACE FUNCTION hr_day9.dept_avg_salary (
53      p_department_id IN hr_day9.departments.department_id%TYPE
54  ) RETURN NUMBER
55  IS
56      l_avg hr_day9.employees.salary%TYPE;
57  BEGIN
58      SELECT AVG(salary)
59      INTO   l_avg
60      FROM   hr_day9.employees
61      WHERE  department_id = p_department_id;
62
63      RETURN ROUND(l_avg, 2);
64  EXCEPTION
65      -- AVG over no rows returns NULL rather than raising, so NO_DATA_FOUND
66      -- cannot happen here. An invalid department is still worth rejecting
67      -- loudly instead of returning a NULL the caller may read as zero.
68      WHEN OTHERS THEN
69          RAISE_APPLICATION_ERROR(-20001,
70              'cannot average department ' || p_department_id || ': ' || SQLERRM);
71  END dept_avg_salary;
```

sql/04_plsql.sql PL/SQL Spaces: 4 UTF-8

Figure 14: The same average as a stored function. AVG over no rows returns NULL rather than raising, so NO_DATA_FOUND cannot happen here, and the comment says so rather than leaving a handler that never fires looking like it matters.

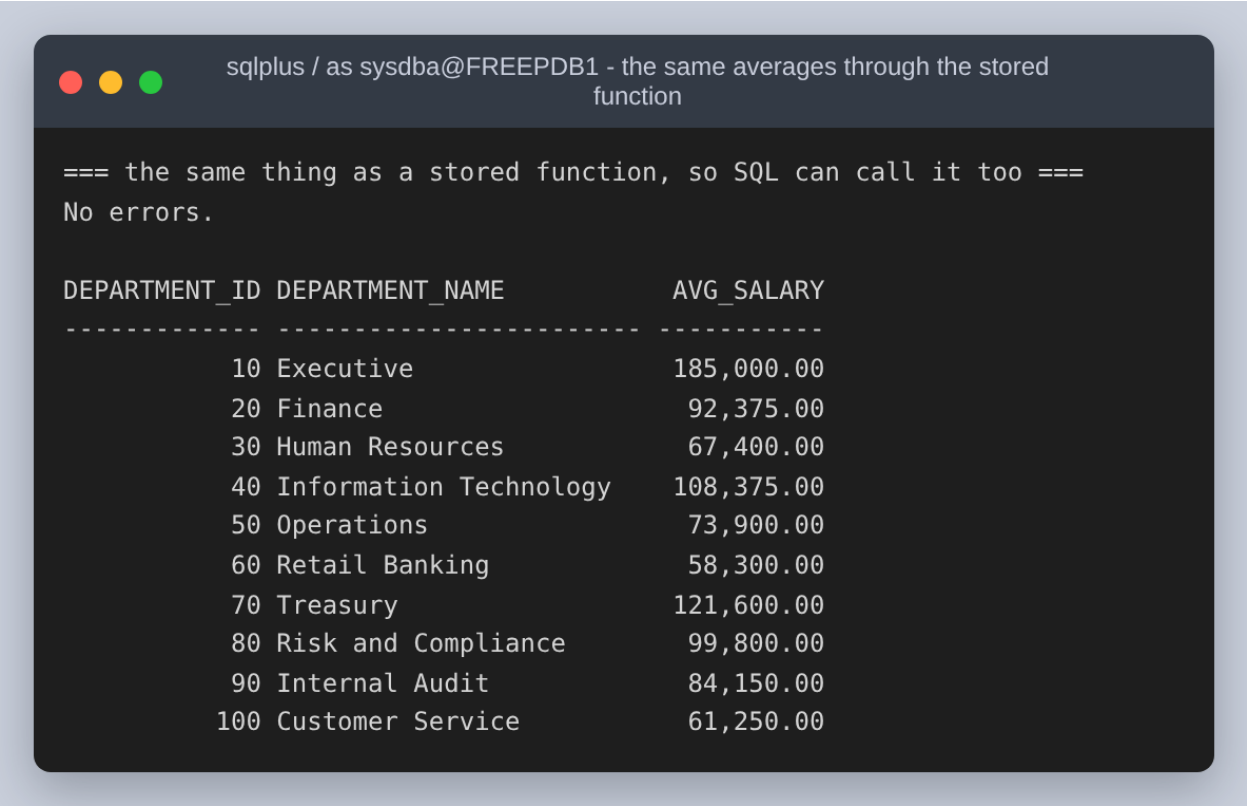

Figure 15: The function called once per department from ordinary SQL, giving the same ten averages.

The plans at twelve rows

The assignment asks for the plan before and after optimization. Taken at twelve rows, that comparison has no content, and it is worth showing why before moving on rather than presenting a meaningless pair of plans as a result.

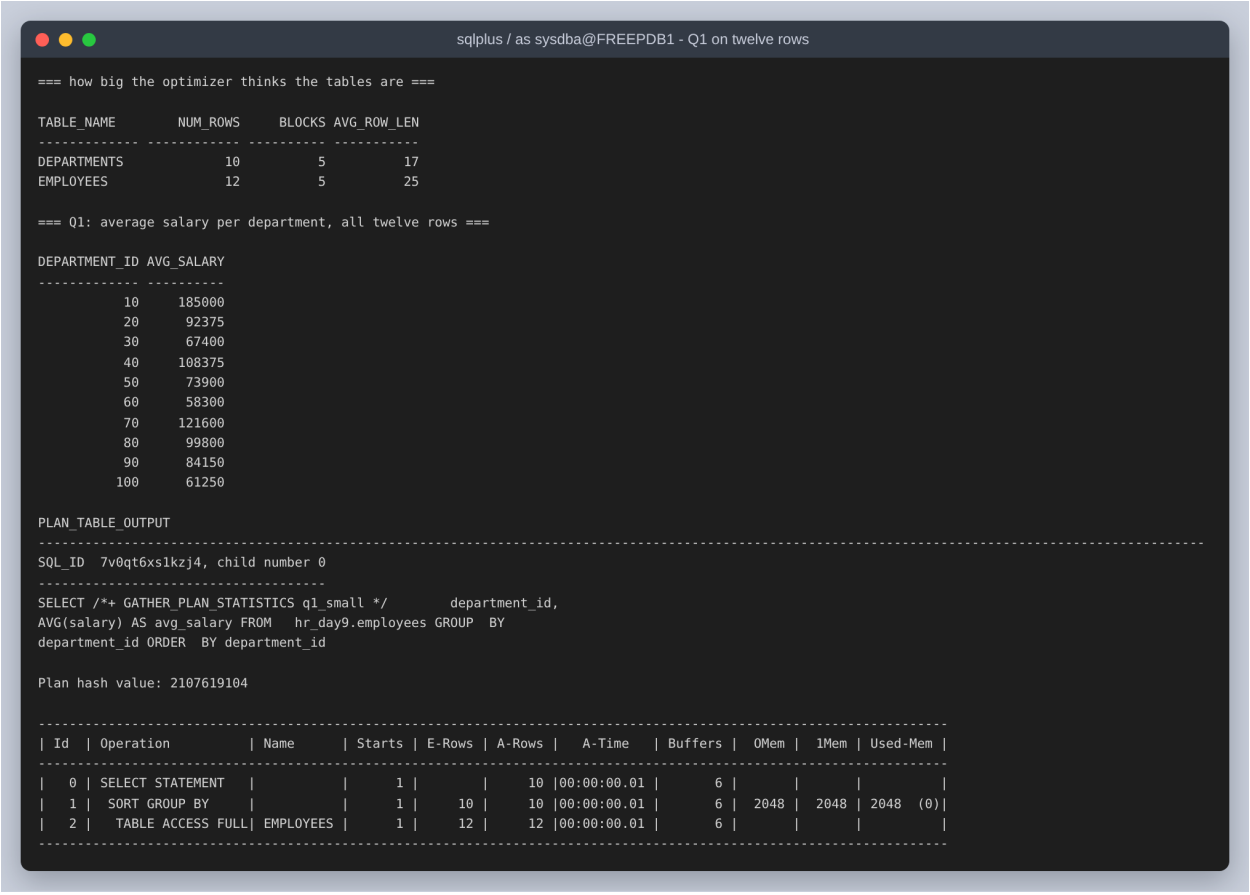

Figure 16: Q1, the average per department, on twelve rows: a full scan costing six buffer gets. The whole table is five blocks, so no index can save more than a few block reads.

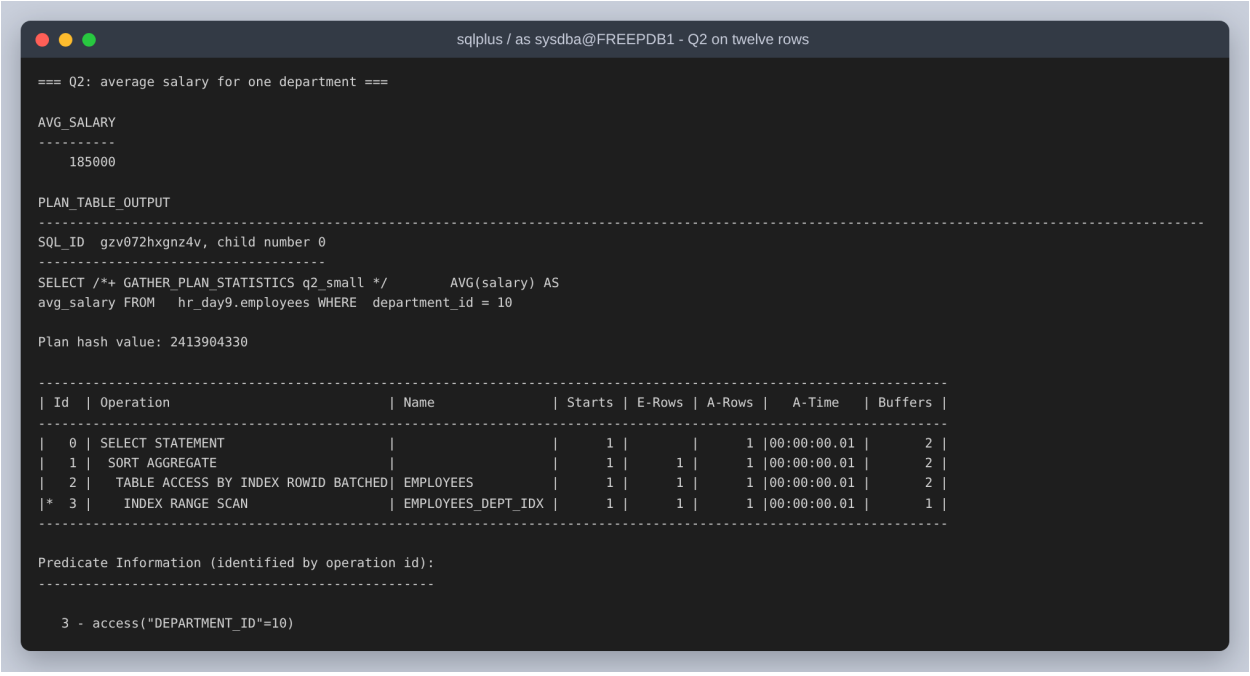

Figure 17: Q2, the average for one department, on twelve rows. Here the optimizer does pick the index, and reads two blocks instead of six. A saving of four block reads is real and irrelevant.

Both plans are correct and neither is interesting. An index is a trade: a second structure to maintain on every insert, in exchange for reading less on the way in. At five blocks there is nothing to read less of. To say anything about optimization the table has to be big enough for the two plans to cost different amounts.

Making the table big enough for the question to have an answer

200,000 more employees are loaded into the same table, across the same ten departments, with the distribution skewed the way a real company is skewed: one small department and nine large ones. That skew is the point. An index helps when it can rule most of the table out, so the interesting query is the one that asks about the small department.

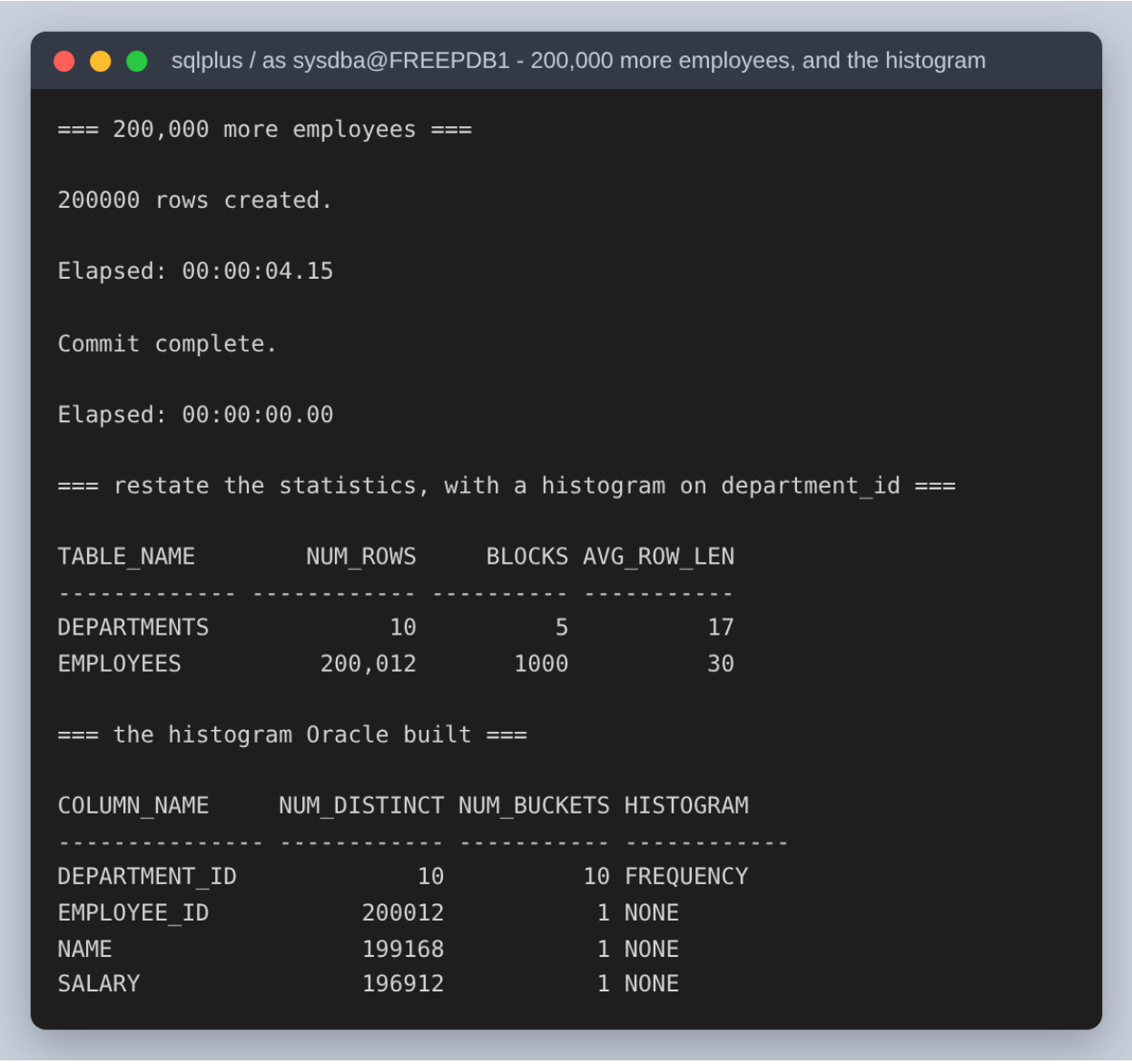

Figure 18: The load, and the statistics gathered afterwards. Oracle built a FREQUENCY histogram with one bucket per department on department_id.

The histogram is what makes the rest of Step 2 work. Without one the optimizer assumes the ten departments are the same size and costs a lookup on any single department at a tenth of the table, which is too much to be worth an index. The histogram tells it that department 10 is 0.25 percent of the rows, and that is the difference between a full scan and an index range scan.

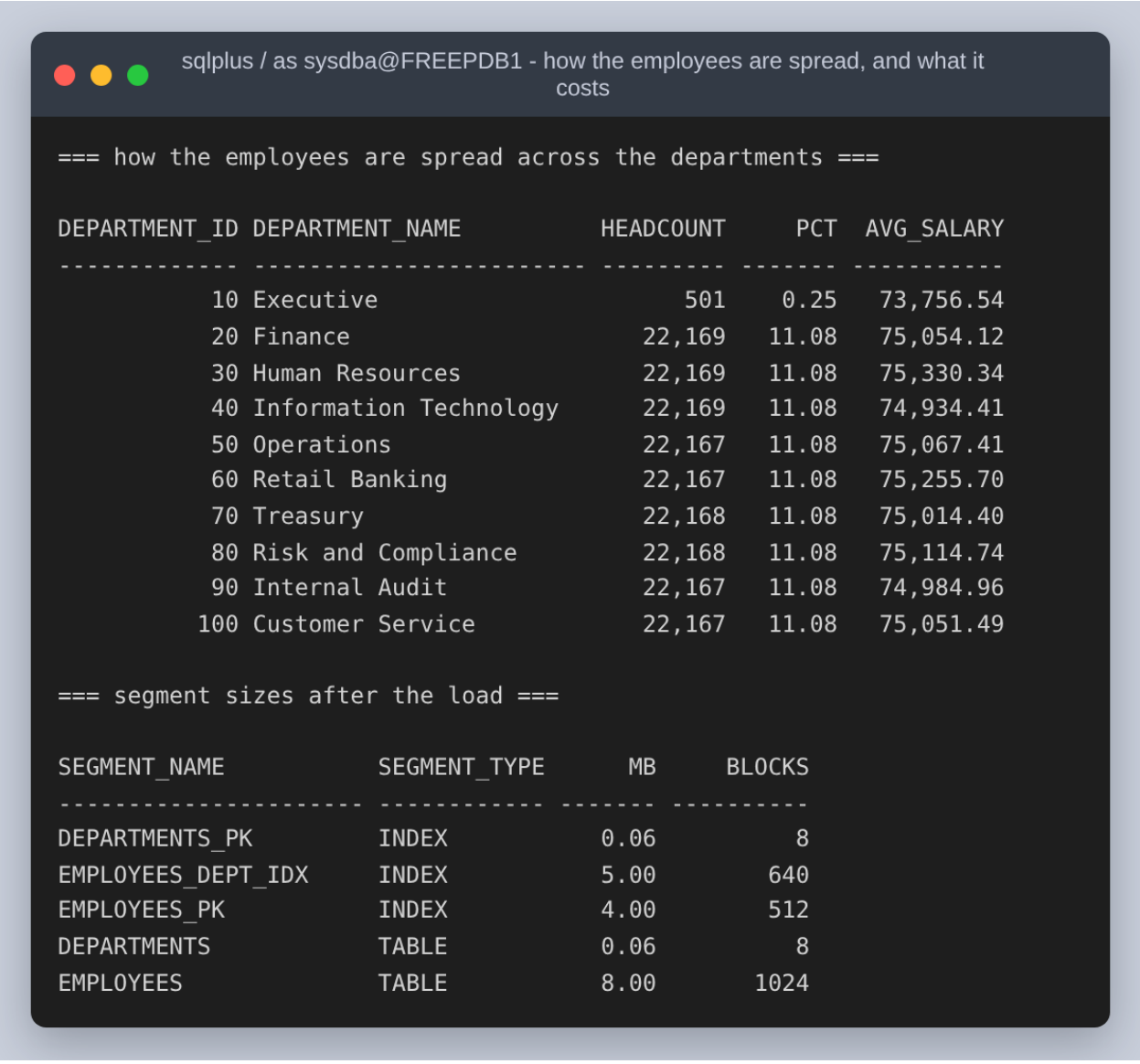

Figure 19: The real distribution: 501 employees in Executive against roughly 22,200 in each of the other nine, and what the segments now cost. The index on department_id alone is already 5 MB against the table's 8 MB.

Before and after, on the same table

To measure the query without the index, the index is made INVISIBLE rather than dropped. Oracle keeps maintaining an invisible index but hides it from the optimizer, so the before and after plans are taken against a byte-for-byte identical table with identical statistics. Dropping and recreating would also rebuild the segment, and then the comparison would have two variables in it instead of one.

```
07_explain_before.sql

13  ALTER INDEX hr_day9.employees_dept_idx INVISIBLE;

sql/07_explain_before.sql   PL/SQL                                  Spaces: 4   UTF-8
```

Figure 20: One statement is the whole of the before/after apparatus.

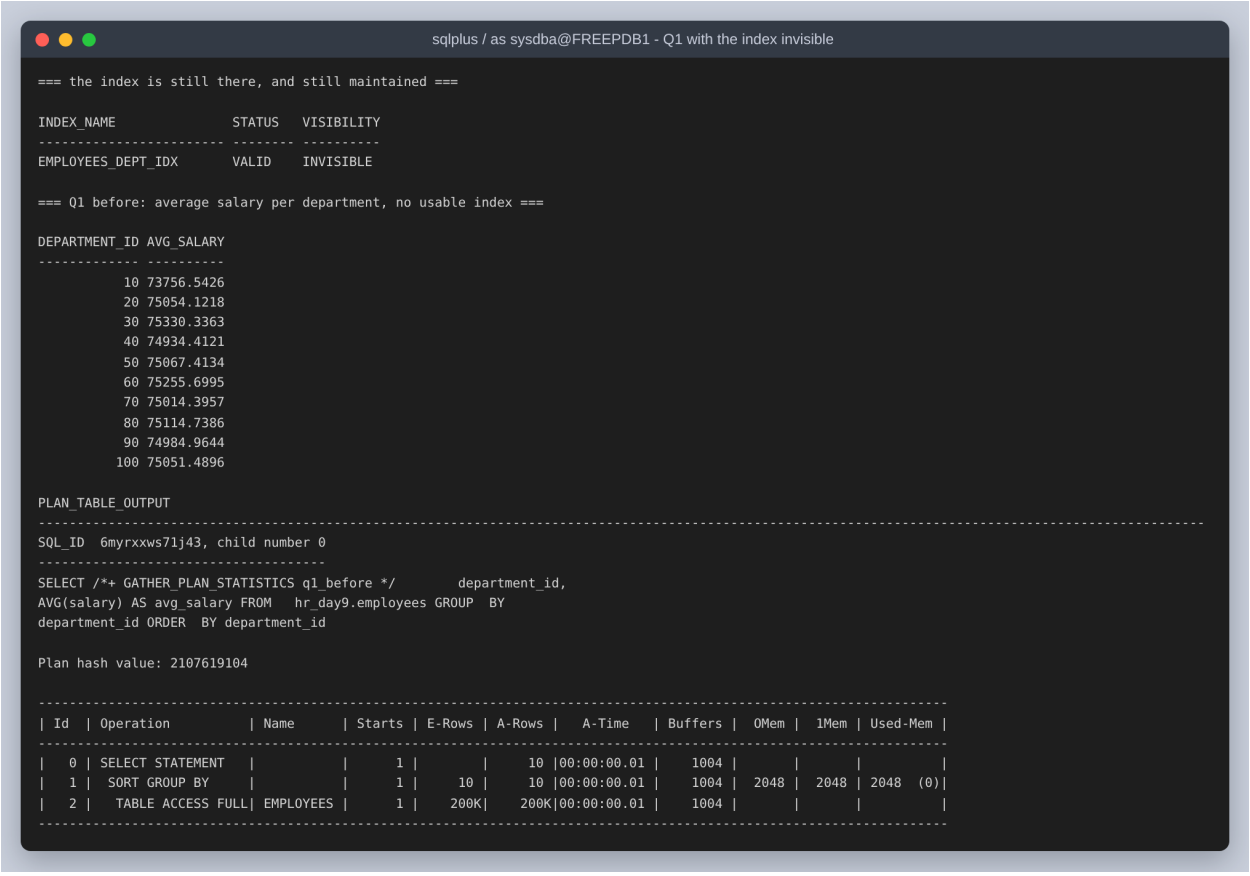

Figure 21: Before, Q1: TABLE ACCESS FULL, 1,006 buffer gets.

sqlplus / as sysdba@FREEPDB1 - Q2 with the index invisible

```
=== Q2 before: average salary for department 10, no usable index ===

AVG_SALARY
----------
73756.5426

PLAN_TABLE_OUTPUT
------------------------------------------------------------------------------------------------------------------------------------------------------
SQL_ID  dx8bd3sug86un, child number 0
-------------------------------------
SELECT /*+ GATHER_PLAN_STATISTICS q2_before */       AVG(salary) AS
avg_salary FROM   hr_day9.employees WHERE  department_id = 10

Plan hash value: 1756381138

-----------------------------------------------------------------------------------------
| Id  | Operation          | Name      | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |           |      1 |        |      1 |00:00:00.01 |    1004 |
|   1 |  SORT AGGREGATE    |           |      1 |      1 |      1 |00:00:00.01 |    1004 |
|*  2 |   TABLE ACCESS FULL| EMPLOYEES |      1 |    501 |    501 |00:00:00.01 |    1004 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - filter("DEPARTMENT_ID"=10)
```

Figure 22: Before, Q2: also TABLE ACCESS FULL, 1,004 buffer gets, with the department filter applied to every row it reads.

sqlplus / as sysdba@FREEPDB1 - Q1 with the index visible

```
=== the index is visible to the optimizer again ===

INDEX_NAME               STATUS   VISIBILITY
------------------------ -------- ----------
EMPLOYEES_DEPT_IDX       VALID    VISIBLE

=== Q1 after: average salary per department, index available ===

DEPARTMENT_ID AVG_SALARY
------------- ----------
           10 73756.5426
           20 75054.1218
           30 75330.3363
           40 74934.4121
           50 75067.4134
           60 75255.6995
           70 75014.3957
           80 75114.7386
           90 74984.9644
          100 75051.4896

PLAN_TABLE_OUTPUT
------------------------------------------------------------------------------------------------------------------------------------------------------
SQL_ID  879jhqnw86tbn, child number 0
-------------------------------------
SELECT /*+ GATHER_PLAN_STATISTICS q1_after */       department_id,
AVG(salary) AS avg_salary FROM   hr_day9.employees GROUP  BY
department_id ORDER  BY department_id

Plan hash value: 2107619104

-------------------------------------------------------------------------------------------------------------
| Id  | Operation          | Name      | Starts | E-Rows | A-Rows |   A-Time   | Buffers |  OMem |  1Mem | Used-Mem |
-------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |           |      1 |        |     10 |00:00:00.01 |    1004 |       |       |          |
|   1 |  SORT GROUP BY     |           |      1 |     10 |     10 |00:00:00.01 |    1004 |  2048 |  2048 | 2048  (0)|
|   2 |   TABLE ACCESS FULL| EMPLOYEES |      1 |    200K|    200K|00:00:00.01 |    1004 |       |       |          |
-------------------------------------------------------------------------------------------------------------
```

Figure 23: After, Q1: the index is visible and the plan has not changed. Same operation, same plan hash value, same 1,006 buffer gets.

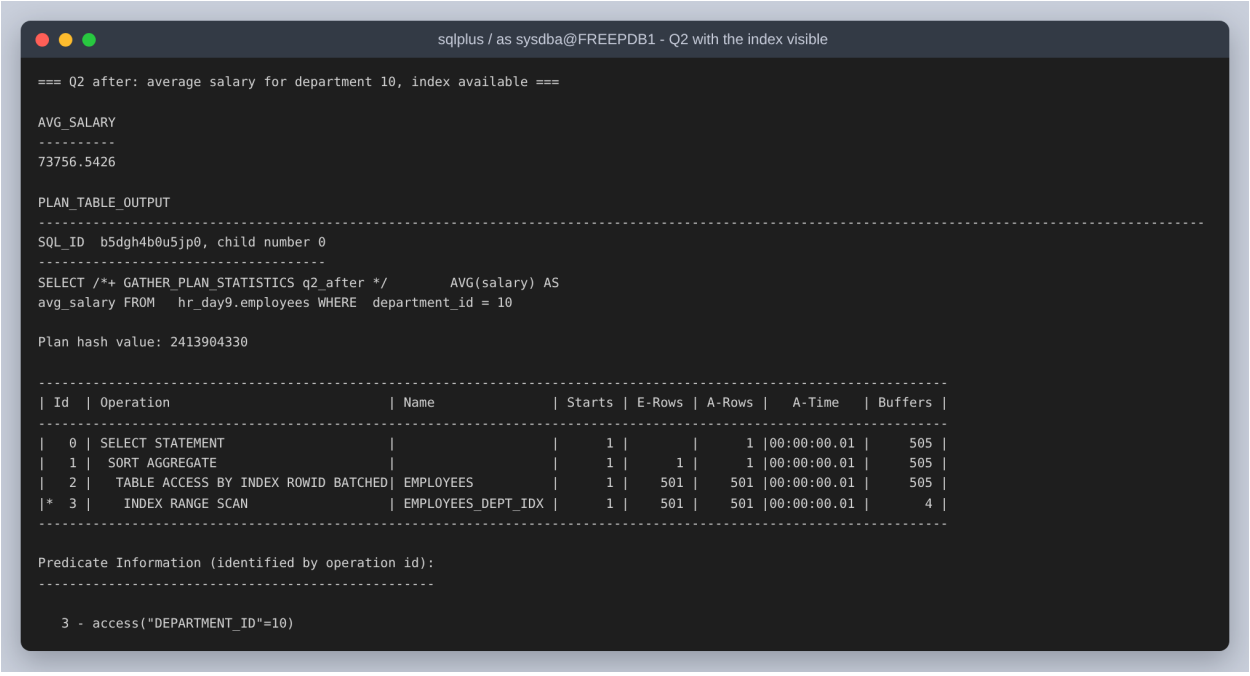

Figure 24: After, Q2: INDEX RANGE SCAN feeding TABLE ACCESS BY INDEX ROWID BATCHED. The index itself costs four gets; the row lookups bring the total to 505.

Query	Before	After	Buffer gets
Q1 - average per department	TABLE ACCESS FULL	TABLE ACCESS FULL	1,006 to 1,006
Q2 - average for department 10	TABLE ACCESS FULL	INDEX RANGE SCAN	1,004 to 505

Q1 is unchanged, and that is not a failure of the index or of the optimizer. Q1 averages the salary of every employee, so it has to read every employee. An index on department_id can find rows by department, but it does not contain the salaries, so any plan built on it would still have to visit all 200,012 table rows to get them, one at a time through a rowid lookup, which is strictly more work than reading the table straight through.

That is a claim about cost, so it is worth checking rather than asserting. Hinting the query to use the index anyway shows what the optimizer decided:

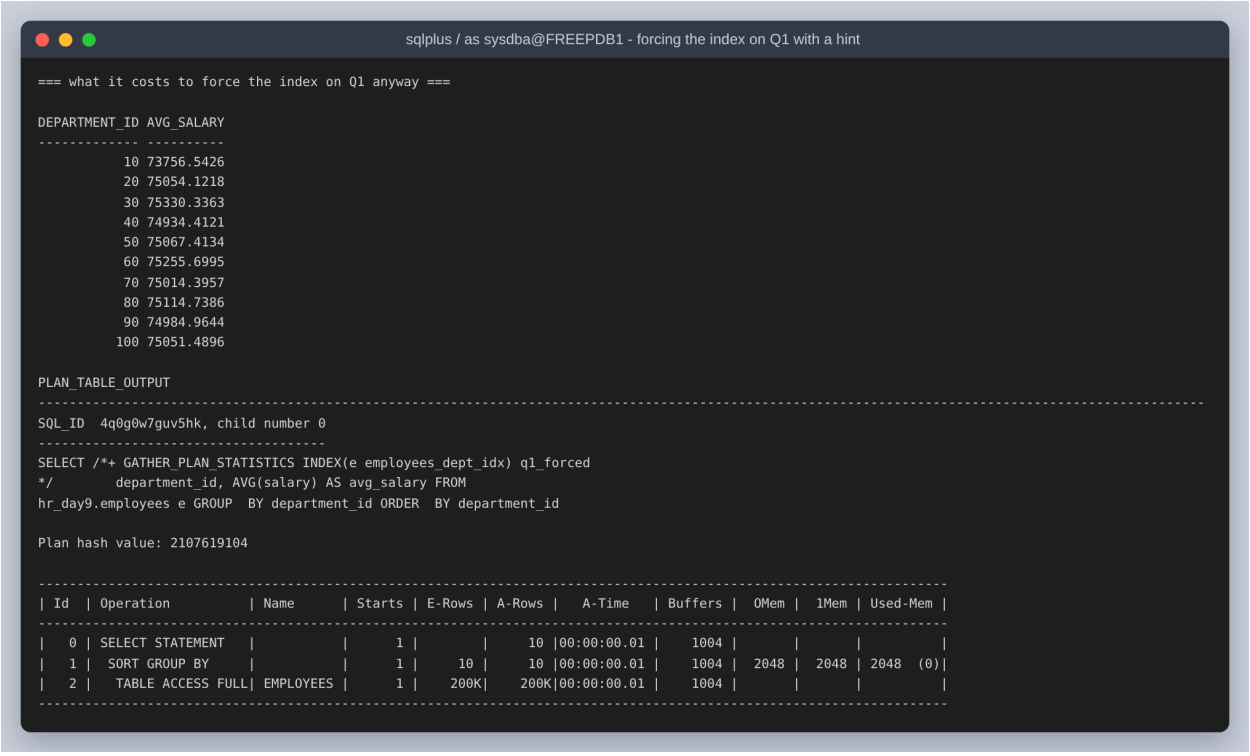

Figure 25: An INDEX hint naming the index on Q1. The plan is the full scan again: the hint is not obeyed, because the index cannot produce the columns the query needs and a hint cannot make a plan legal that was not. A hint that is silently dropped looks exactly like a hint that worked, which is a good reason to read the plan rather than trust the hint.

The optimisation that does help Q1

If the problem with the single-column index is that it does not hold the salaries, the fix is an index that does. An index on (department_id, salary) holds both columns Q1 reads, so the aggregate can be answered from the index and the table never has to be opened. The index is smaller than the table, because it holds two columns instead of four, and reading less is the whole of the improvement.

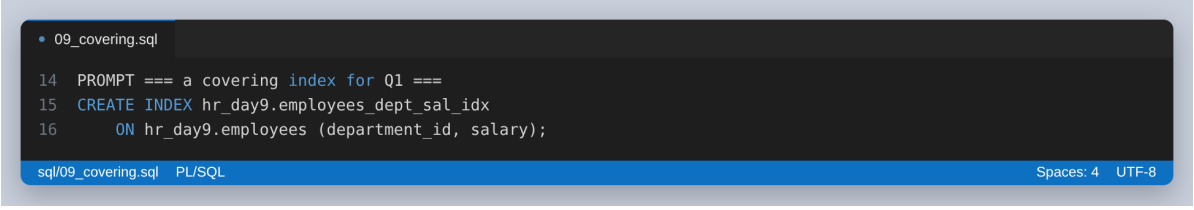

Figure 26: The covering index. The comment records why it is allowed to answer a query over the whole table.

That last point is a real constraint and not a detail. A B-tree index leaves a row out only when every indexed column is NULL, so an index on a nullable column alone cannot be trusted to contain every row. Here salary is NOT NULL, which guarantees an entry for every employee, and that guarantee is what lets the optimizer answer a GROUP BY over the whole table from the index. Had salary been nullable, this optimisation would not have been available.

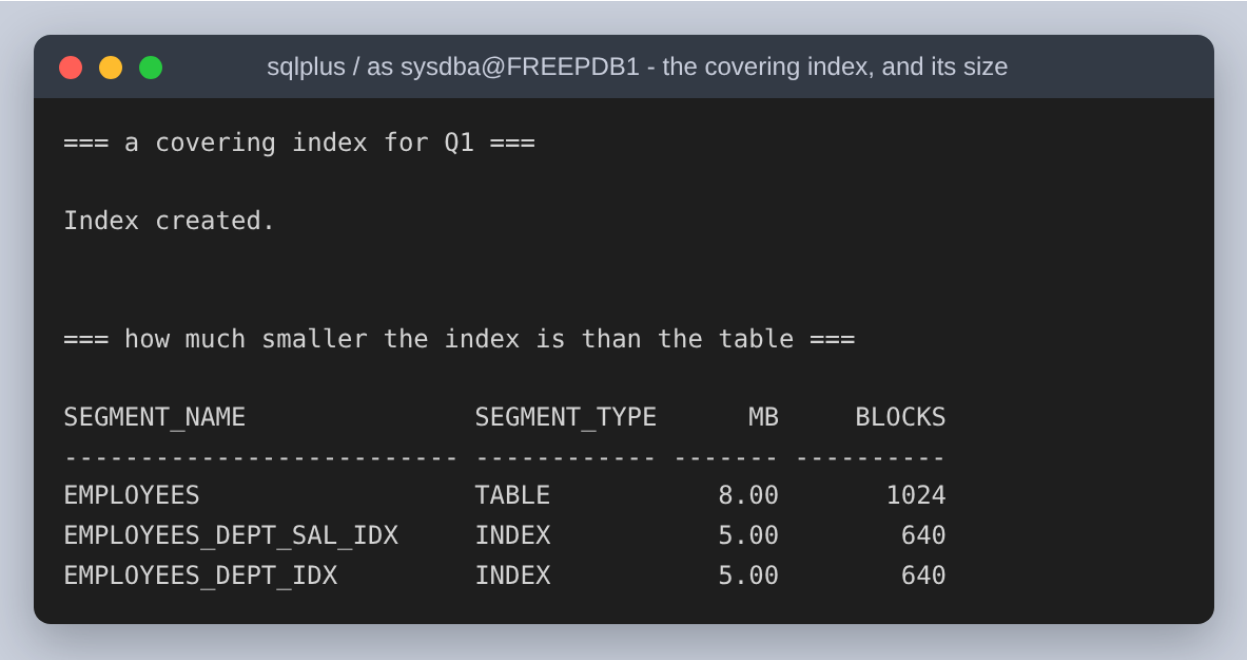

Figure 27: The covering index being created, and the sizes side by side.

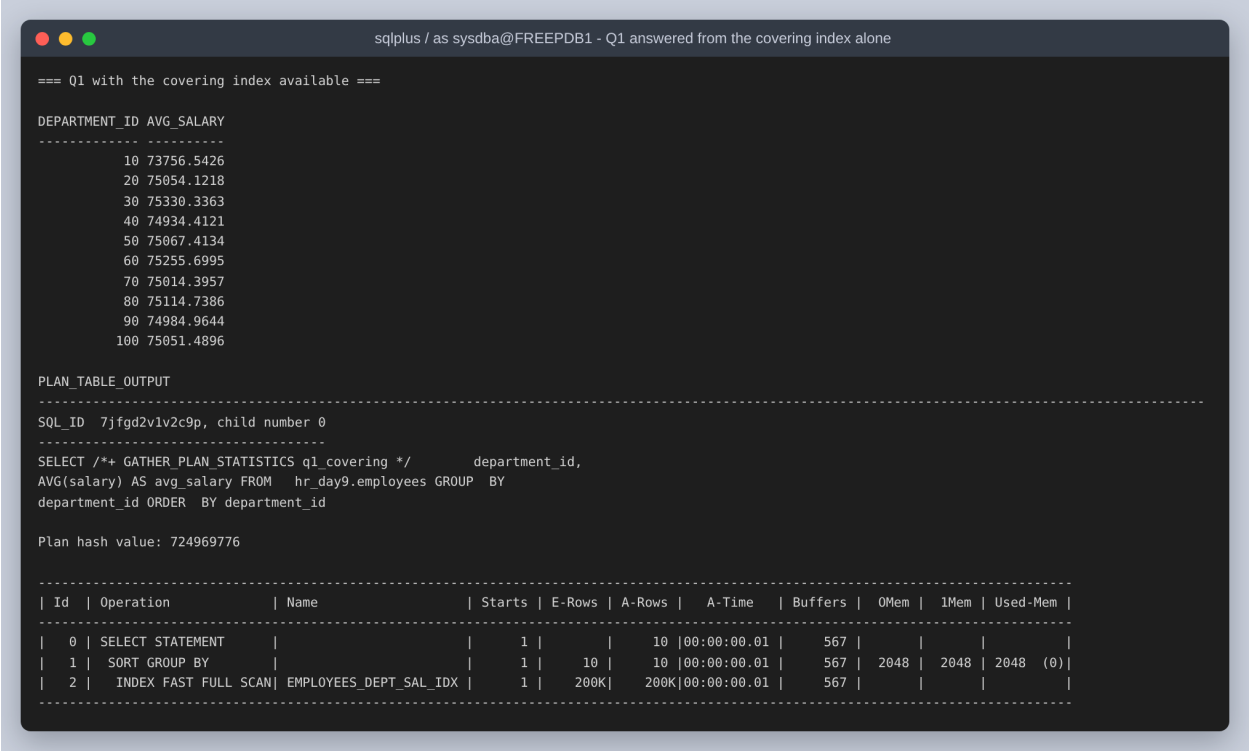

Figure 28: Q1 answered by INDEX FAST FULL SCAN with no table access at all: 569 buffer gets against 1,006, a 43 percent reduction.

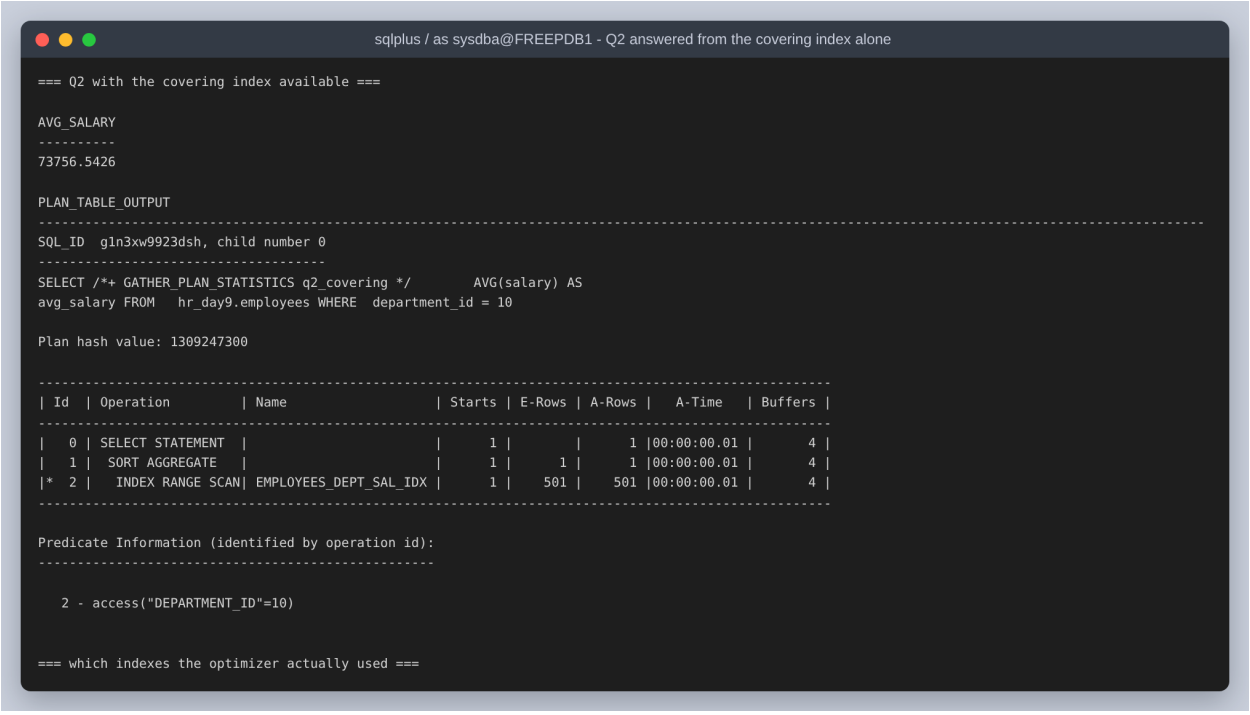

Figure 29: Q2 with the same index: 4 buffer gets. The index range scan now finds the salaries in the index beside the department, so the 501 rowid lookups into the table disappear.

Measured, rather than only planned

Plans predict cost; they do not prove it. Each configuration was therefore run in a loop, Q1 fifty times and Q2 five hundred times, with the logical reads read back out of v$sqlstats per cursor.

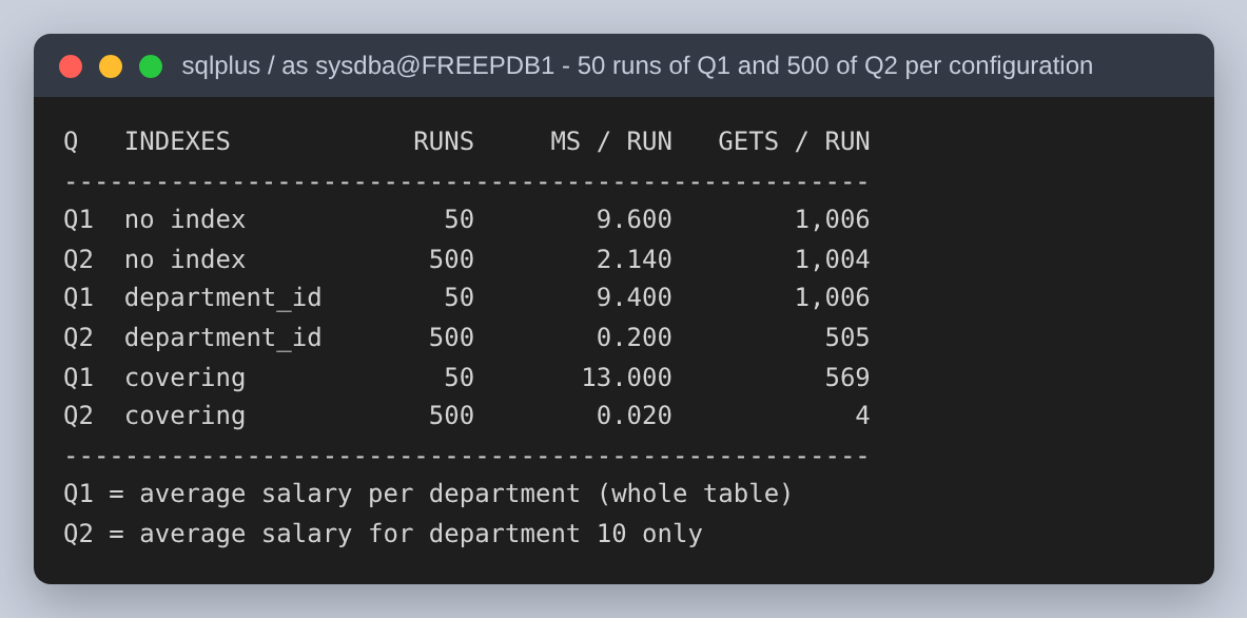

Figure 30: Fifty runs of Q1 and five hundred of Q2 under each of the three index configurations.

	No index	department_id	(department_id, salary)
Q1 buffer gets	1,006	1,006	569
Q1 ms per run	9.600	9.400	13.000
Q2 buffer gets	1,004	505	4
Q2 ms per run	2.140	0.200	0.020

Q2 is the clear win: 10.7x faster with the index the assignment asks for, and 107.0x faster with the covering index, which is what reading four blocks instead of a thousand looks like on the clock.

> Q1 is more interesting, because the two measurements disagree. The covering index cuts Q1's logical reads by 43 percent, from 1,006 to 569 blocks, and yet Q1 got slower: 13.000 ms against 9.600 ms. Fewer blocks is not the same as less time. A full table scan is read with multiblock reads, while an index fast full scan of a 200,000-entry index walks more, smaller structures; at this size the table already fits in the buffer cache, so there is no physical I/O for the block saving to save, and only the extra CPU is left. The block reduction is what would pay off on a table too large to cache. Had only the plans been collected, this would have been written up as a straight improvement.

Step 3: backup and recovery

The image ships in NOARCHIVELOG mode, and in that mode the online redo logs are overwritten as they fill. That leaves no record of what changed after a backup, so the only recovery possible is back to the moment of the backup, and only from a backup taken while the database was shut down. ARCHIVELOG mode copies each redo log off before it is reused. It is what makes both a backup of an open database and a recovery to a chosen point in time possible, and this assignment needs both.

sqlplus / as sysdba - the bounce through MOUNT that enables ARCHIVELOG

```
=== before: the log mode as the image ships it ===

LOG_MODE
------------
NOARCHIVELOG

1 row selected.


=== where the archived logs and backups will go ===

System altered.


System altered.


=== a clean bounce through MOUNT, which is where the switch is made ===
Database closed.
Database dismounted.
ORACLE instance shut down.
ORACLE instance started.

Total System Global Area 1603392928 bytes
Fixed Size                  5027232 bytes
Variable Size             587202560 bytes
Database Buffers         1006632960 bytes
Redo Buffers                4530176 bytes
Database mounted.

Database altered.


Database altered.


Pluggable database altered.
```

Figure 31: The switch to ARCHIVELOG. It is made at MOUNT, so the database is closed and reopened around it; the flash recovery area is pointed somewhere with room first.

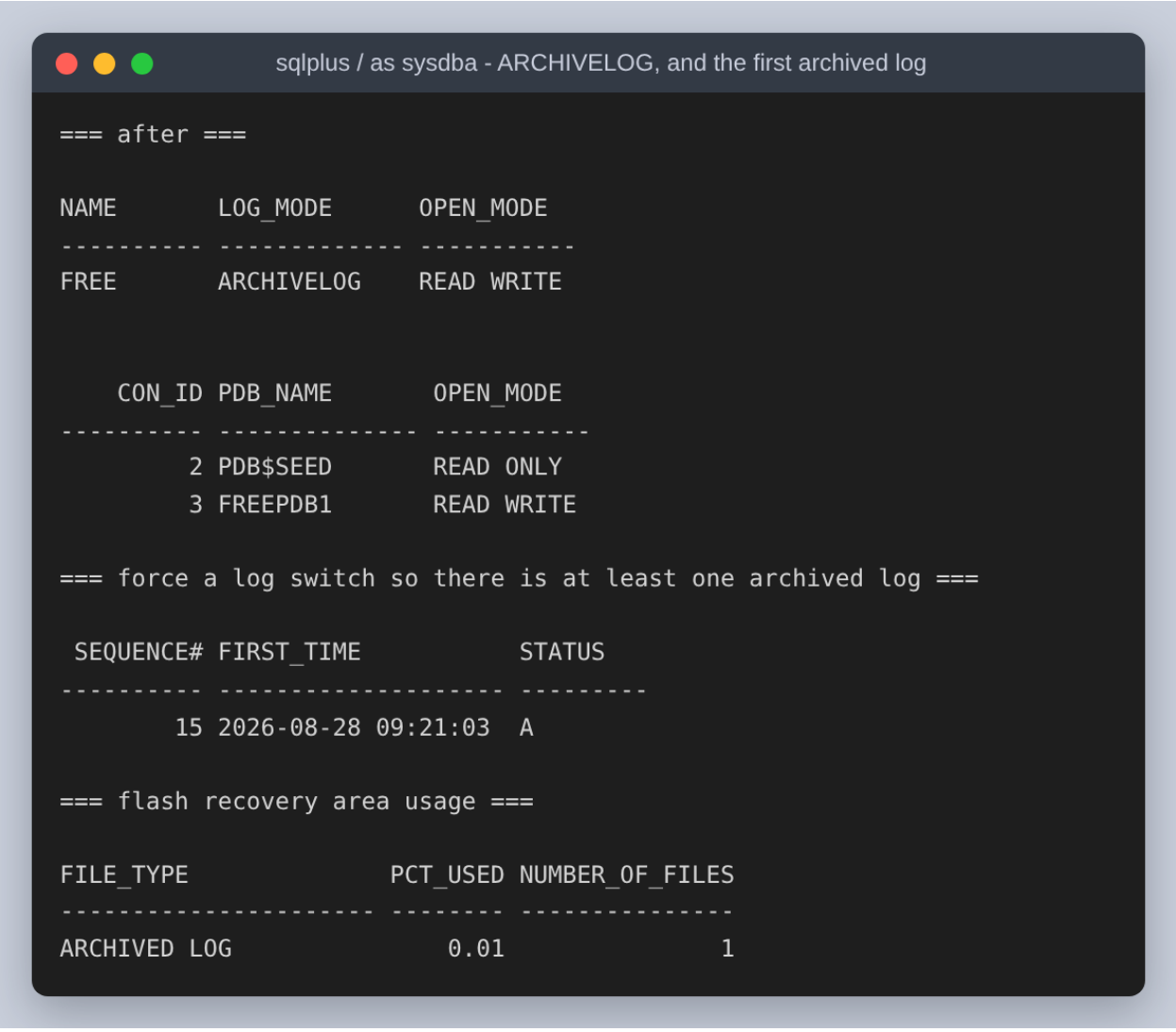

Figure 32: ARCHIVELOG confirmed, both pluggable databases open again, and a forced log switch so that at least one archived log exists before the backup runs.

Before backing anything up, the state being backed up is recorded: the row counts, the total of all 200,012 salaries, a hash fingerprint of the employees table, and the list of objects that will have to come back. Without this, a successful recovery can only be declared, not shown.

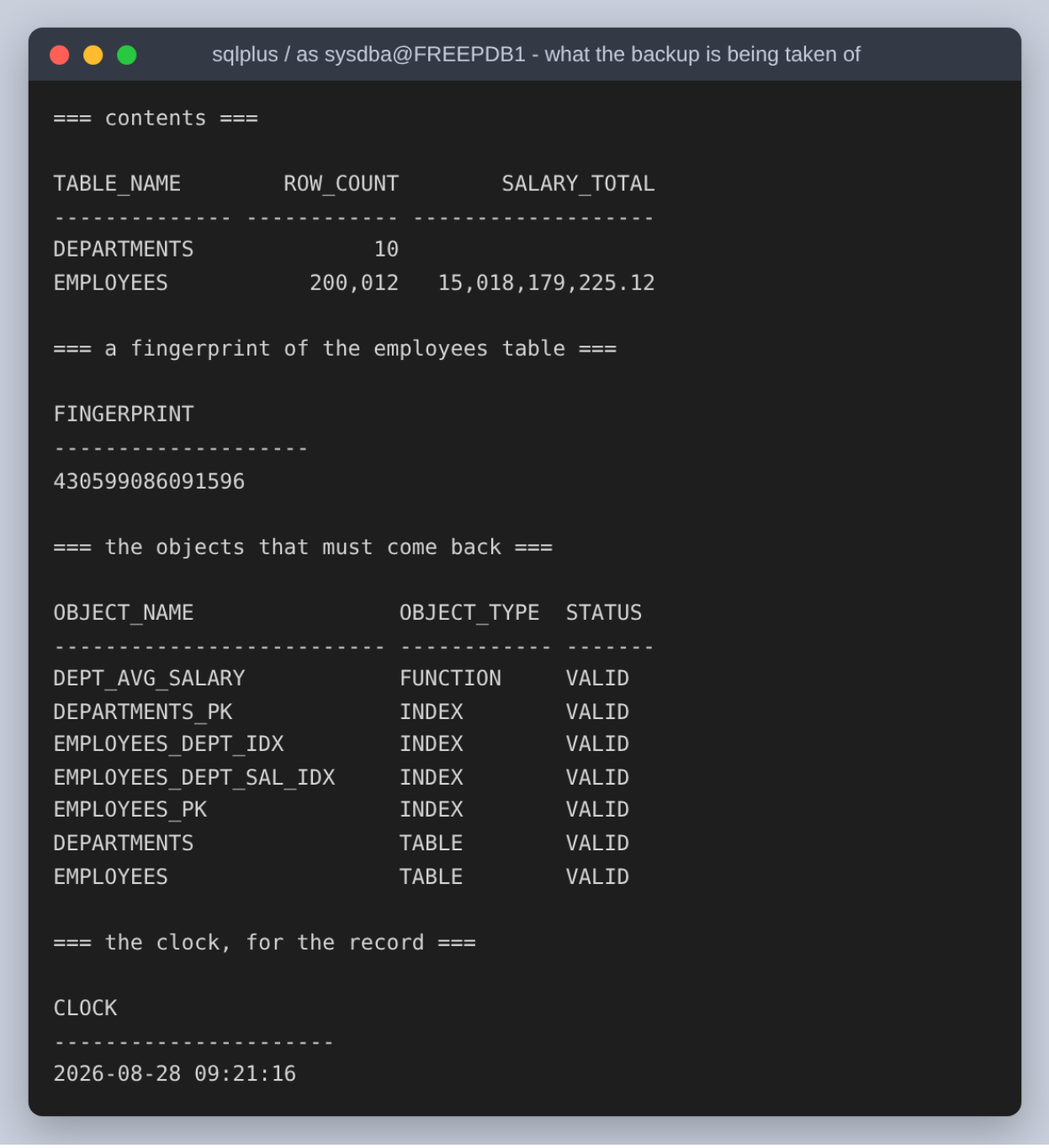

Figure 33: What the backup is a backup of. The fingerprint sums a hash of every employee row: a total of the salaries alone would not notice two employees swapping values, and a hash of the ordered rows would.

The backup

RMAN is Oracle's own backup tool, and for this it is the one that matters, because it understands the file formats. A filesystem copy of an open database catches datafiles mid-write and produces something that cannot be opened. RMAN reads through the instance, checks every block as it goes, and records what it did in the control file so a later RESTORE knows what exists.

01_backup.rman

```
11  SHOW ALL;
12
13  # Two channels because the container has eight cores and the backup is the
14  # slowest step in the assignment.
15  CONFIGURE DEVICE TYPE DISK PARALLELISM 2;
16
17  # With autobackup on, RMAN writes a copy of the control file and the spfile at
18  # the end of every backup. Without it a recovery that has lost the control file
19  # has to be talked through the backup set by hand.
20  CONFIGURE CONTROLFILE AUTOBACKUP ON;
21
22  CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF 7 DAYS;
23
24  # What is being backed up, before backing it up.
25  REPORT SCHEMA;
26
27  # PLUS ARCHIVELOG archives the current redo log first, includes the archived
28  # logs in the backup, and archives again afterwards. That is what makes the
29  # backup usable on its own: the datafile copies are inconsistent, because the
30  # database was open while they were read, and the redo in the same backup is
31  # what reconciles them.
32  BACKUP AS COMPRESSED BACKUPSET DATABASE PLUS ARCHIVELOG;
33
34  LIST BACKUP SUMMARY;
```

rman/01_backup.rman Shell Script Spaces: 4 UTF-8

Figure 34: The backup script. PLUS ARCHIVELOG is the part that makes the backup usable on its own.

PLUS ARCHIVELOG archives the current redo log first, includes the archived logs in the backup, then archives again afterwards. The datafile copies inside the backup are inconsistent with each other, because the database was open and changing while they were read, and the redo in the same backup is what reconciles them.

rman target / - the configuration the backup runs under

```
CONFIGURE SNAPSHOT CONTROLFILE NAME TO '/opt/oracle/product/26ai/dbhomeFree/dbs/snapcf_FREE.f'; # default

new RMAN configuration parameters:
CONFIGURE DEVICE TYPE DISK PARALLELISM 2 BACKUP TYPE TO BACKUPSET;
new RMAN configuration parameters are successfully stored

new RMAN configuration parameters:
CONFIGURE CONTROLFILE AUTOBACKUP ON;
new RMAN configuration parameters are successfully stored

new RMAN configuration parameters:
CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF 7 DAYS;
new RMAN configuration parameters are successfully stored
```

Figure 35: The configuration the backup runs under. Control file autobackup is turned on so that a recovery which has also lost the control file does not have to be talked through the backup set by hand.

rman target / - REPORT SCHEMA: every file in the backup

```
Report of database schema for database with db_unique_name FREE

List of Permanent Datafiles
===========================
File Size(MB) Tablespace           RB segs Datafile Name
---- -------- -------------------- ------- ----------------------
1    930      SYSTEM               YES     /opt/oracle/oradata/FREE/system01.dbf
2    288      PDB$SEED:SYSTEM      NO      /opt/oracle/oradata/FREE/pdbseed/system01.dbf
3    594      SYSAUX               NO      /opt/oracle/oradata/FREE/sysaux01.dbf
4    373      PDB$SEED:SYSAUX      NO      /opt/oracle/oradata/FREE/pdbseed/sysaux01.dbf
7    10       USERS                NO      /opt/oracle/oradata/FREE/users01.dbf
18   11       UNDOTBS1             YES     /opt/oracle/oradata/FREE/undotbs01.dbf
20   11       PDB$SEED:UNDOTBS1    NO      /opt/oracle/oradata/FREE/pdbseed/undotbs01.dbf
21   298      FREEPDB1:SYSTEM      YES     /opt/oracle/oradata/FREE/FREEPDB1/system01.dbf
22   373      FREEPDB1:SYSAUX      NO      /opt/oracle/oradata/FREE/FREEPDB1/sysaux01.dbf
23   41       FREEPDB1:UNDOTBS1    YES     /opt/oracle/oradata/FREE/FREEPDB1/undotbs01.dbf
24   31       FREEPDB1:USERS       NO      /opt/oracle/oradata/FREE/FREEPDB1/users01.dbf
```

Figure 36: REPORT SCHEMA: every datafile in the container database and in both pluggable databases, which is what BACKUP DATABASE will cover.

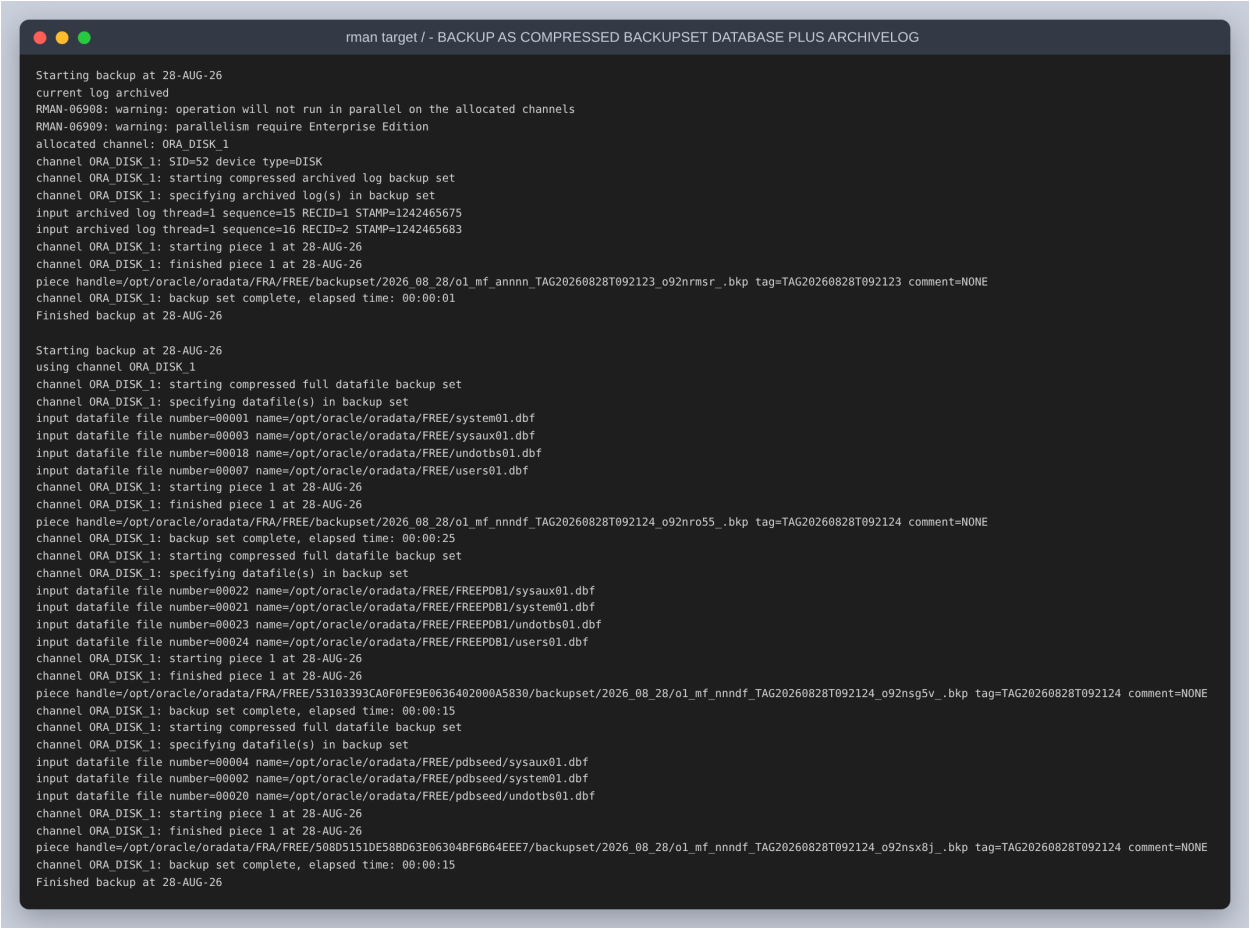

Figure 37: The backup itself: the archived logs, then all fifteen datafiles as compressed backup sets, then the archived logs again.

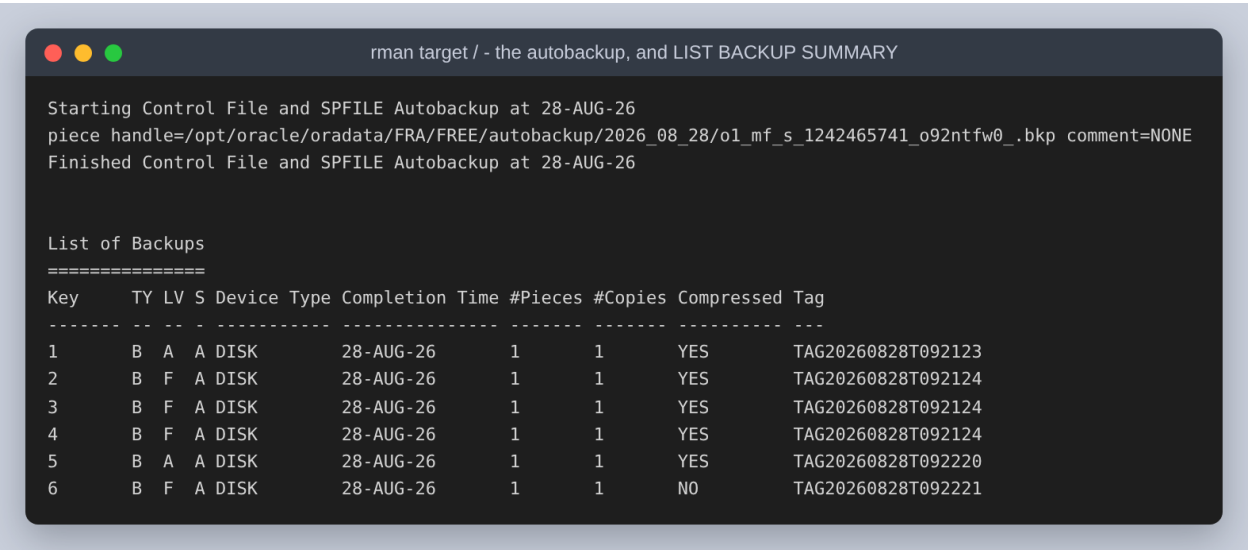

Figure 38: The control file and SPFILE autobackup, and LIST BACKUP SUMMARY showing the six backup sets that resulted.

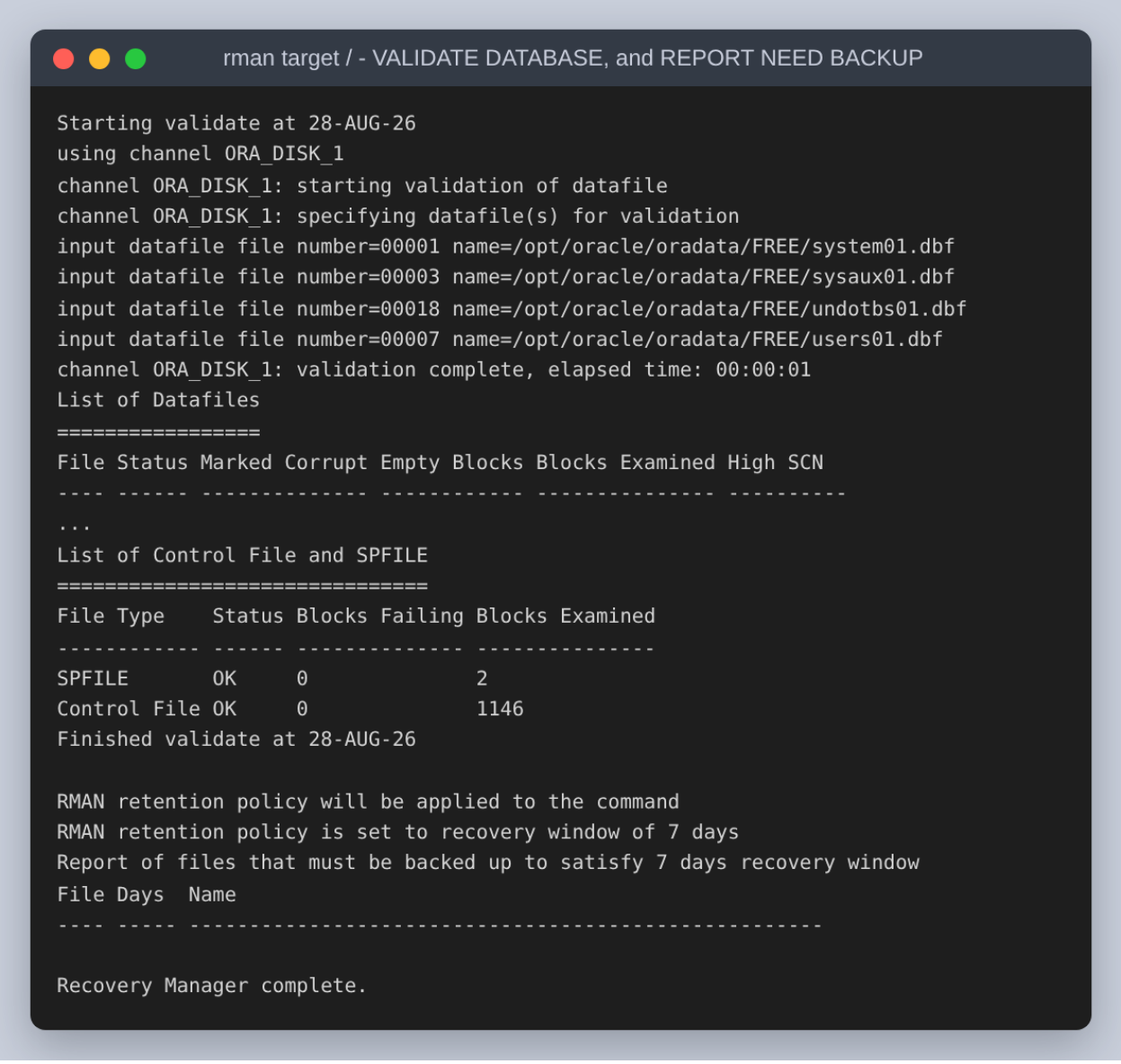

Figure 39: VALIDATE DATABASE reads every block of every file and checks it, so that the recovery is not the first time anyone finds out whether the backup is sound. Zero blocks marked corrupt, and REPORT NEED BACKUP lists nothing outstanding.

One honest note on the configuration. The script asks for PARALLELISM 2, and RMAN answers with RMAN-06908 and RMAN-06909: parallel backup is an Enterprise Edition feature, so Free allocates one channel and says so. The warning is visible in the figures above and in the restore later on. It is left in rather than tidied away, because the setting is harmless and the warning is the evidence for which edition this actually ran on.

The failure

The failure is a dropped employees table, with PURGE. That keyword is deliberate. A plain DROP TABLE moves the segment to the recycle bin, and then the recovery is one FLASHBACK TABLE away and the backup is never touched. PURGE removes that shortcut, which is the point: the assignment asks for a recovery from the backup, so the failure has to be one the backup is the only answer to.

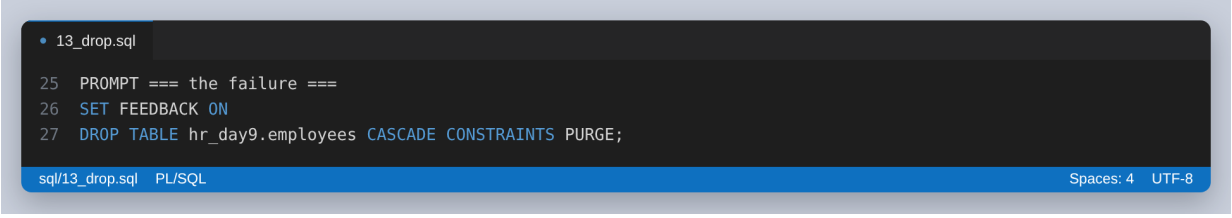

Figure 40: The failure, and the SCN read one statement before it.

The SCN is Oracle's own logical clock. Every commit gets a higher one, so recovering until a given SCN means putting the database back the way it was at that instant. It is read one statement before the DROP and passed straight to RMAN by setup.sh, so the number in the recovery command is that number rather than a retyped copy of it.

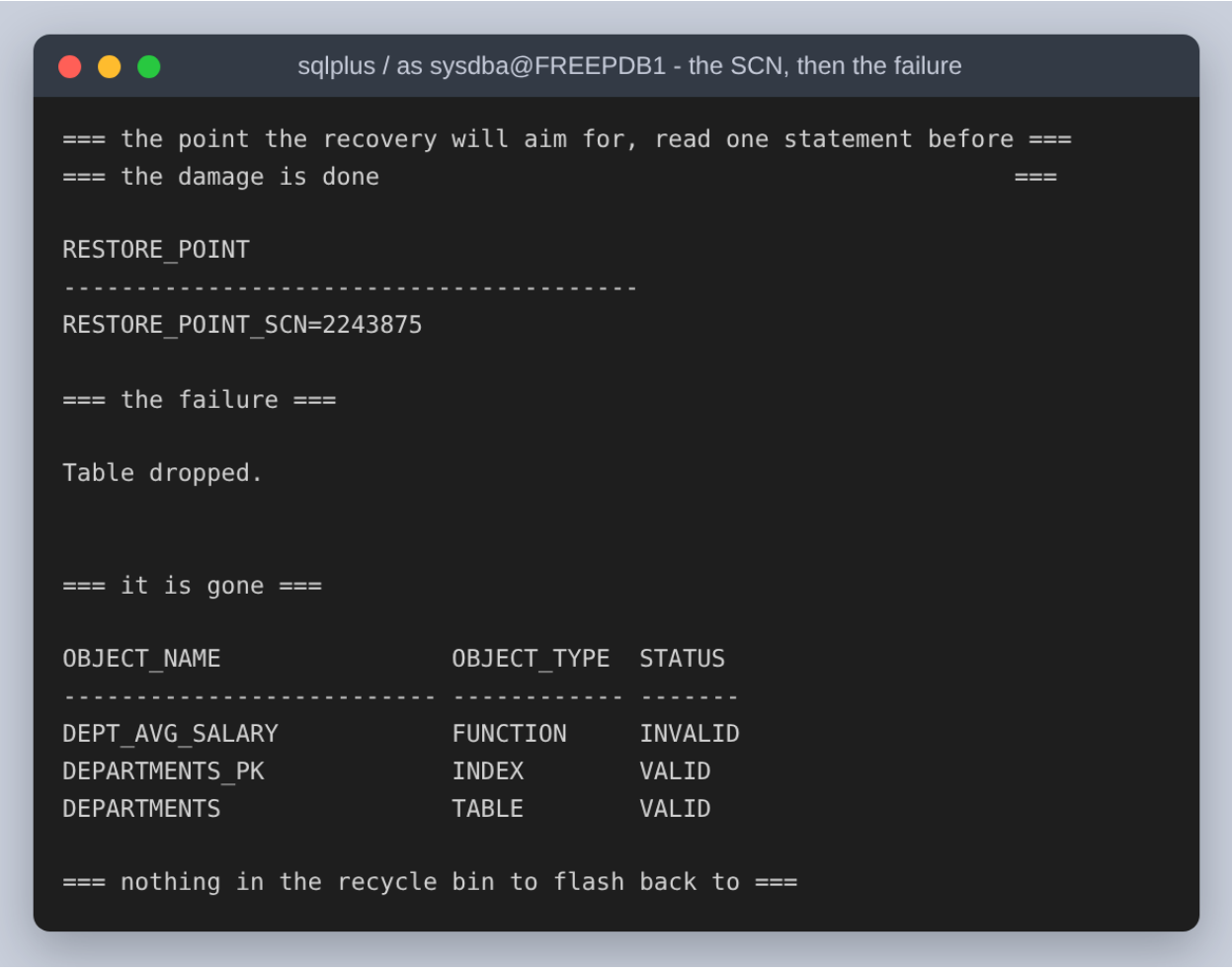

Figure 41: The restore point, SCN 2243875, then the table dropped. Afterwards HR_DAY9 holds only departments and its index, and the stored function has gone INVALID because the table it reads is gone.

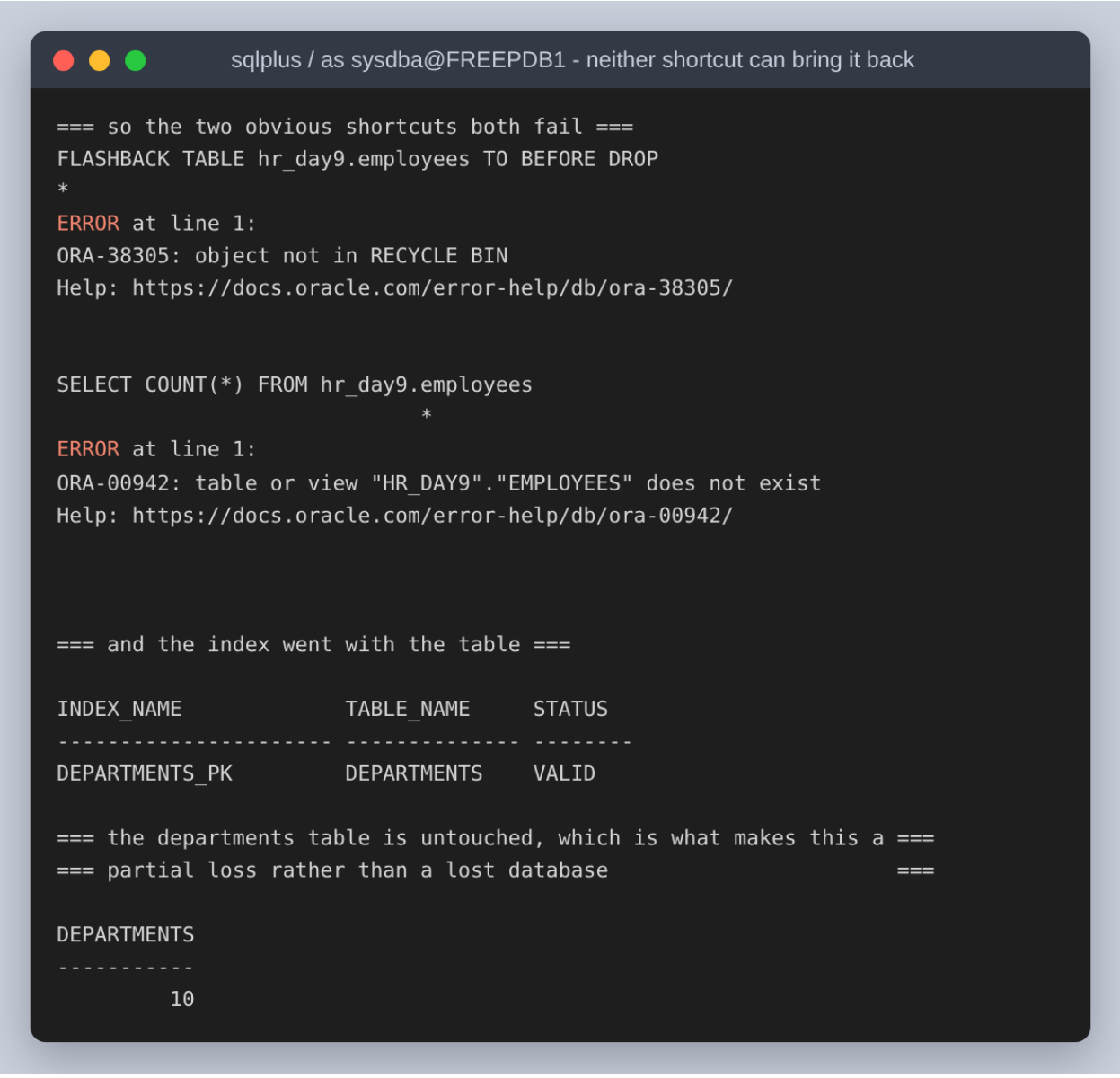

Figure 42: Both shortcuts refusing. FLASHBACK TABLE TO BEFORE DROP fails with ORA-38305 because PURGE left nothing in the recycle bin, and selecting from the table fails with ORA-00942. departments survives untouched, which makes this a partial loss rather than a lost database, and a partial loss is the harder case: the recovery has to bring one table back without rolling the rest of the schema somewhere else.

The recovery

This is a point-in-time recovery of the whole container database to the SCN captured before the DROP. RESTORE copies the datafiles back out of the backup, in the inconsistent state they were read in. RECOVER replays the archived redo logs on top of them up to the SCN, and that is the step that turns the copies into a working database with the employees table in it.

02_restore.rman

```
RUN {
    # RESTORE needs the datafiles closed but the control file readable, and
    # MOUNT is the state where both are true.
    SHUTDOWN IMMEDIATE;
    STARTUP MOUNT;

    # SET UNTIL applies to both commands below: it stops RESTORE from picking a
    # backup taken after the target, and stops RECOVER from applying redo past
    # it. Setting it once is what keeps the two consistent.
    SET UNTIL SCN &1;

    # RESTORE copies the datafiles back out of the backup. They come back as
    # they were during the backup, which is mid-transaction and unusable.
    RESTORE DATABASE;

    # RECOVER replays the archived redo logs on top of them, up to the SCN, and
    # that is the step that turns the copies into a consistent database. The
    # employees table exists again at the end of this line.
    RECOVER DATABASE;

    ALTER DATABASE OPEN RESETLOGS;
}

LIST INCARNATION OF DATABASE;
```

rman/02_restore.rman Shell Script Spaces: 4 UTF-8

Figure 43: The recovery script. SET UNTIL is set once and governs both commands, which is what keeps RESTORE from picking a backup taken after the target and RECOVER from applying redo past it.

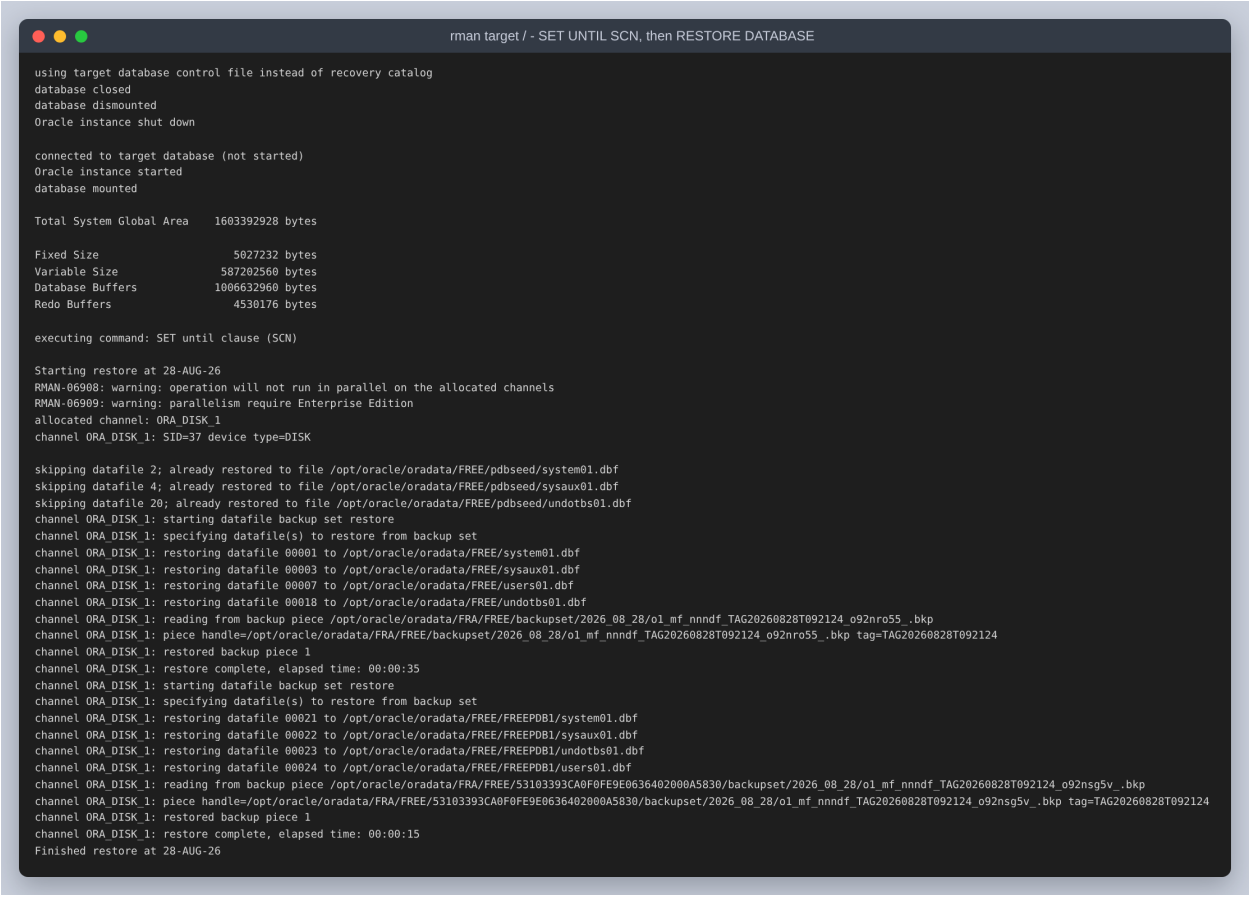

Figure 44: SHUTDOWN, STARTUP MOUNT, the SET UNTIL SCN taking effect, and all the datafiles being restored from the compressed backup sets.

The database then has to be opened with RESETLOGS. The redo written after the chosen SCN, which includes the DROP itself, is being deliberately abandoned, so the log sequence cannot continue past a point the datafiles no longer agree with and has to start over.

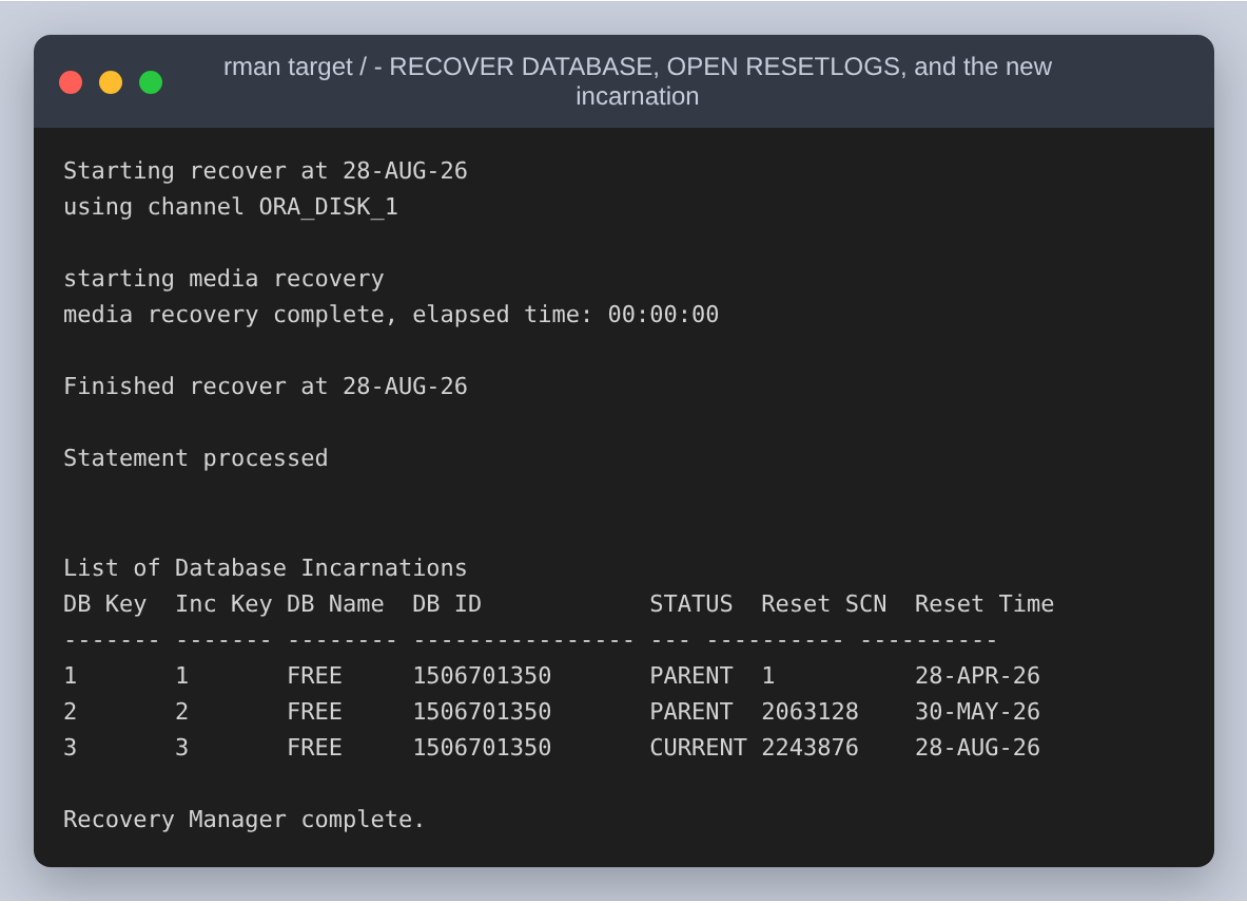

Figure 45: RECOVER DATABASE, then ALTER DATABASE OPEN RESETLOGS. LIST INCARNATION shows a third incarnation created at SCN 2243876, one past the 2243875 that was asked for, which is the database confirming where it was put back to.

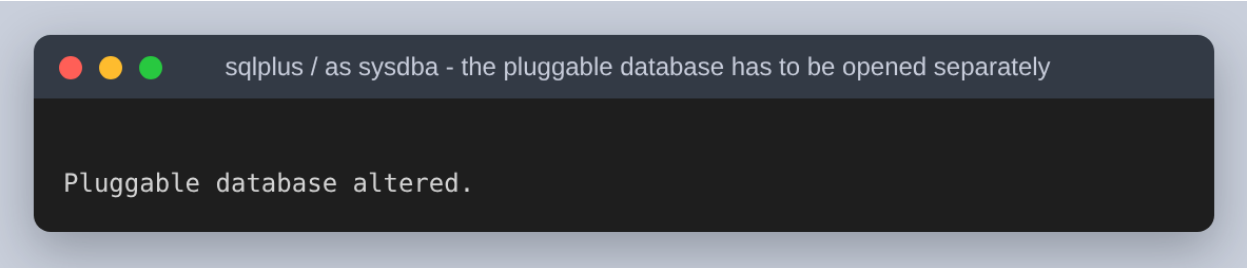

Figure 46: OPEN RESETLOGS opens the container database and leaves the pluggable databases closed, so FREEPDB1 is opened separately.

Checking the recovery instead of declaring it

The queries from before the backup are now run again, and compared.

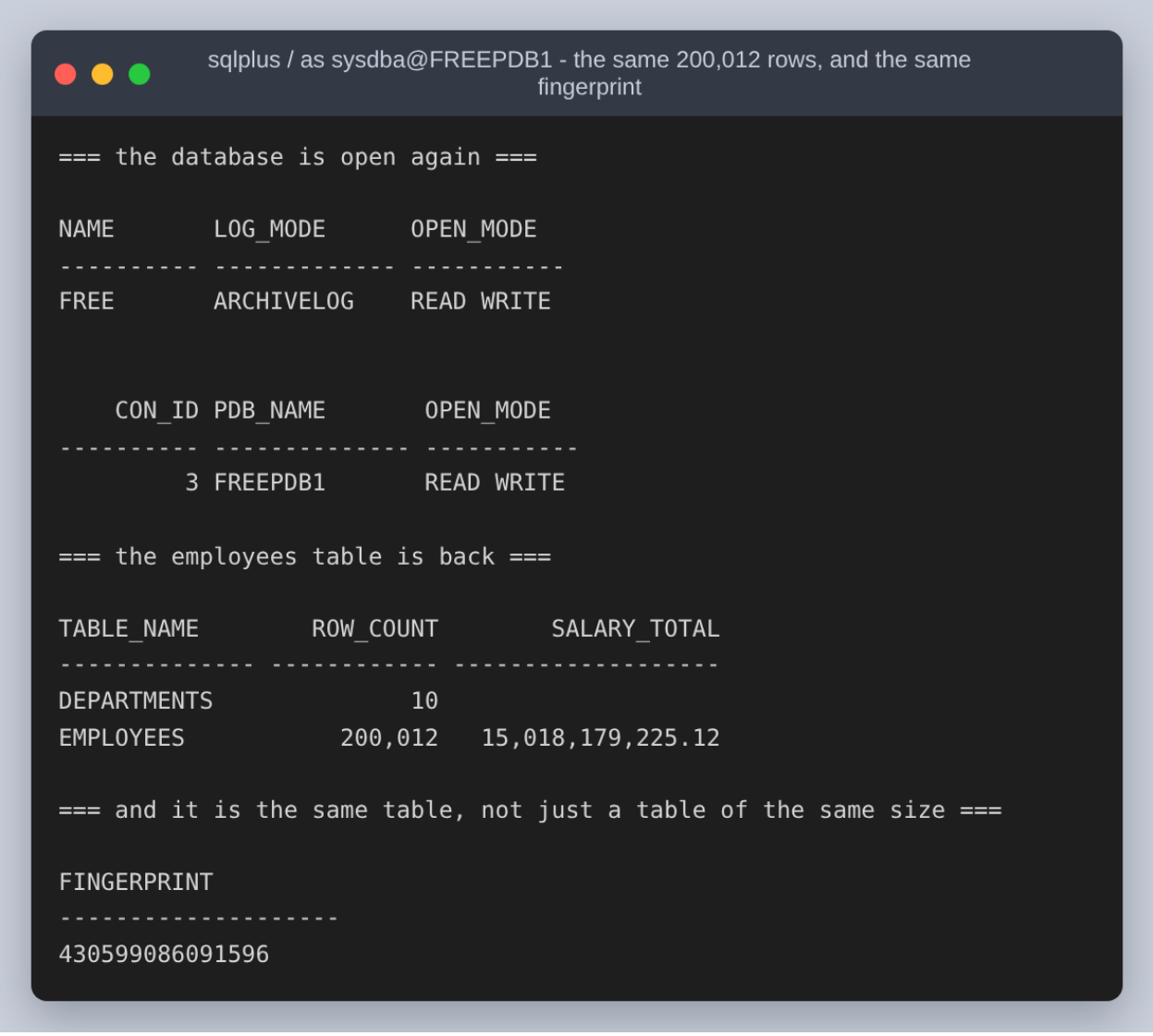

Figure 47: The same 200,012 rows, the same salary total of 15,018,179,225.12, and the same fingerprint 430599086091596. departments still holds its ten rows, so nothing else was rolled backwards to get the employees back.

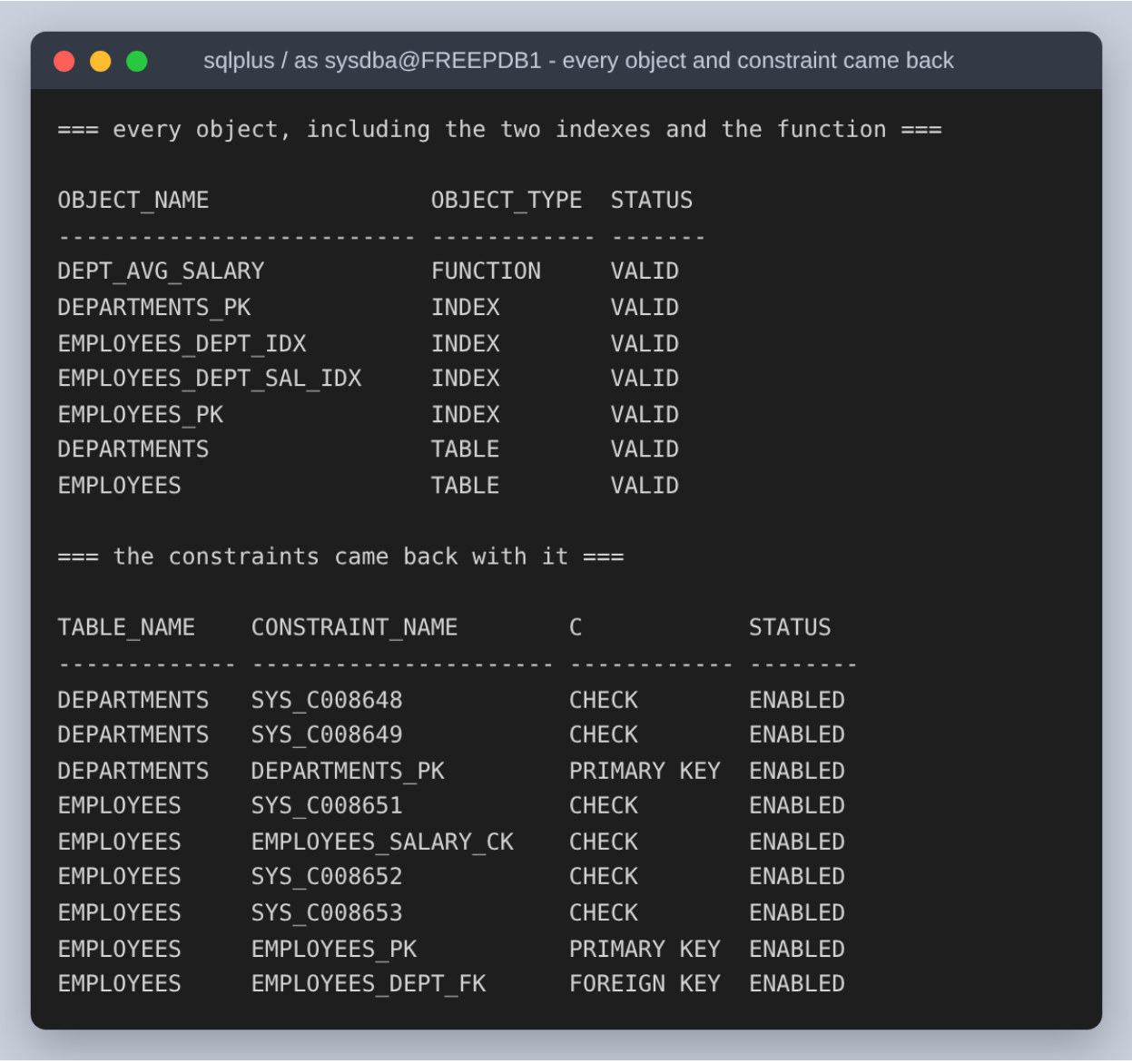

Figure 48: Both indexes, the function and every constraint back and valid, including the foreign key to departments and the check on salary. The recovery restored the schema, not just the data.

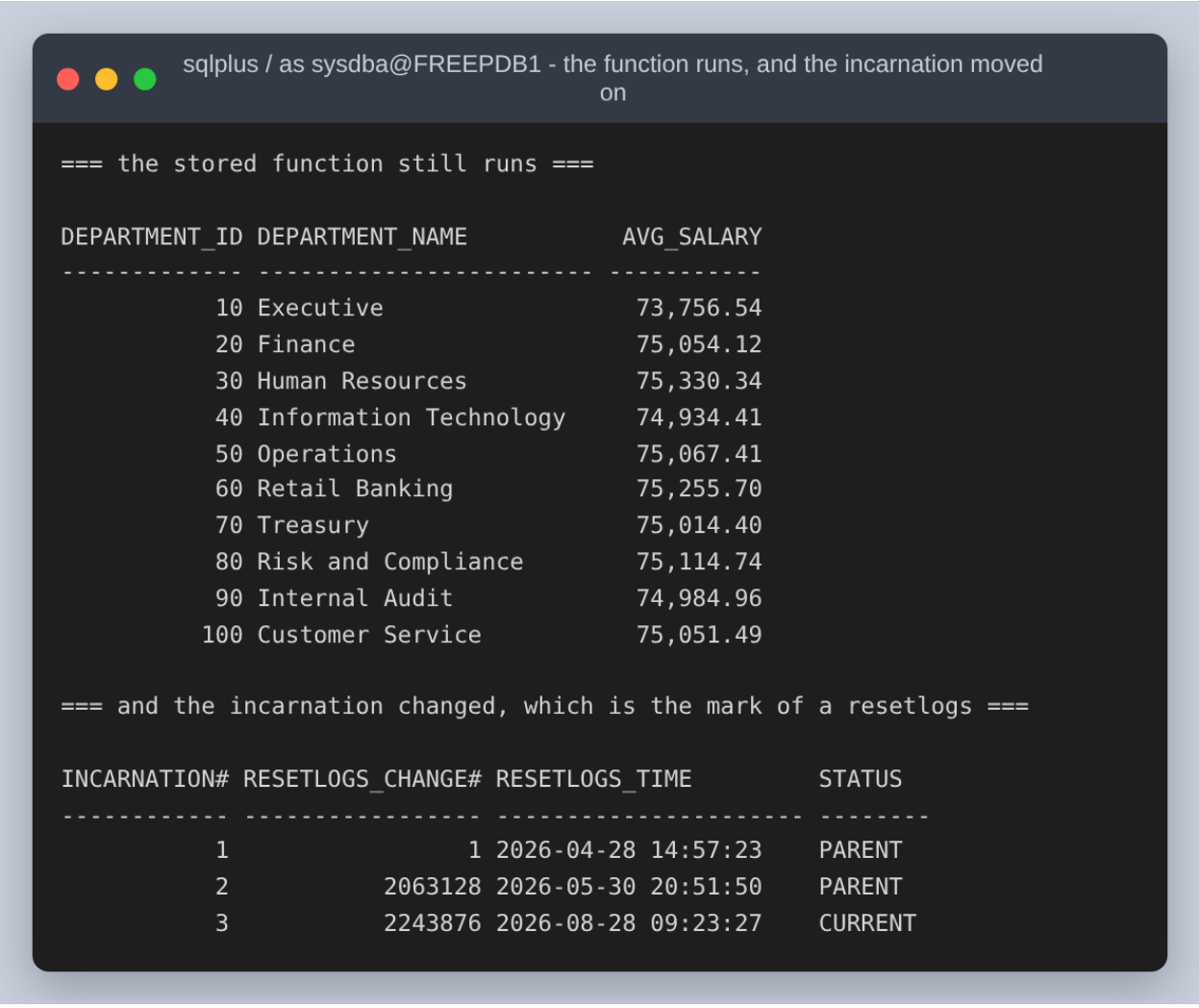

Figure 49: The stored function, INVALID after the drop, recompiling and returning the ten averages again.

	Before the backup	After the recovery
employees rows	200,012	200,012
Sum of salaries	15,018,179,225.12	15,018,179,225.12
Row fingerprint	430599086091596	430599086091596
departments rows	10	10
Objects in HR_DAY9	7 valid	7 valid

The fingerprint matching is the part worth insisting on. Matching row counts would only show that a table of the right size came back. The fingerprint sums a hash of the four columns of every row, so it would change if any single salary came back different, and it did not change.

The optimisations, in one place

Gathered here because the assignment asks for them as explanations rather than as figures.

- An index on employees(department_id). It changes the plan for a query filtering on one department from TABLE ACCESS FULL to INDEX RANGE SCAN, from 1,004 buffer gets to 505, and makes it 10.7x faster. It does nothing at all for the average-per-department query, because that query reads every row by definition.
- A histogram on department_id, gathered with METHOD_OPT FOR COLUMNS SIZE 254. Without it the optimizer assumes the ten departments are equally sized and will not choose the index for any of them. The index is only usable because the statistics describe the skew.
- A covering index on employees(department_id, salary). This is the one that changes the assignment's own query: it becomes an INDEX FAST FULL SCAN with no table access, reading 569 blocks instead of 1,006. It also cuts the single-department query to 4 gets. The measured wall clock for Q1 went up rather than down, and the note in Step 2 explains why.
- Leaving the aggregate in SQL. The PL/SQL formats the result of a GROUP BY rather than fetching rows and averaging them in a loop, which keeps the work in the database instead of moving 200,012 rows across the call interface to compute ten numbers.
- Using INVISIBLE to measure. Not an optimisation of the database, but the reason the before and after numbers are comparable: they were taken against the same segment with the same statistics, with only the optimizer's view of the index changed.

The PL/SQL query used

The cursor at the centre of the anonymous block, quoted from sql/04_plsql.sql. The LEFT JOIN is what makes a department with no employees appear with a NULL average instead of being dropped from the report.

```
    CURSOR c_dept_avg IS
        SELECT   d.department_id,
                 d.department_name,
                 COUNT(e.employee_id)   AS headcount,
                 AVG(e.salary)          AS avg_salary
        FROM     hr_day9.departments d
                 LEFT JOIN hr_day9.employees e
                        ON e.department_id = d.department_id
        GROUP BY d.department_id, d.department_name
        ORDER BY d.department_id;
```

And the stored function, which returns the same average for one department and is the form the covering index helps most:

```
CREATE OR REPLACE FUNCTION hr_day9.dept_avg_salary (
    p_department_id IN hr_day9.departments.department_id%TYPE
) RETURN NUMBER
IS
    l_avg hr_day9.employees.salary%TYPE;
```

```
BEGIN
    SELECT AVG(salary)
    INTO   l_avg
    FROM   hr_day9.employees
    WHERE  department_id = p_department_id;

    RETURN ROUND(l_avg, 2);
EXCEPTION
    -- AVG over no rows returns NULL rather than raising, so NO_DATA_FOUND
    -- cannot happen here. An invalid department is still worth rejecting
    -- loudly instead of returning a NULL the caller may read as zero.
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20001,
            'cannot average department ' || p_department_id || ': ' || SQLERRM);
END dept_avg_salary;
```

Reproducing it

One script does the whole assignment against a fresh instance and writes every capture the figures are built from. It takes about fifteen minutes, most of it the RMAN backup and restore.

```
cd oracle-tuning
./setup.sh                      # instance, schema, tuning, backup, recovery
python3 scripts/make_figures.py  # results/ into figures/
python3 build.py                 # figures/ into the .docx and .pdf
```

setup.sh runs in a single invocation on purpose. The container runtime on the machine this was built on terminates the container when the shell that started it exits, so splitting the steps across separate shells would lose the database between them.

What I would take away from this

- An index is a claim about a query, not about a column. The assignment's index and the assignment's query look like they belong together, and the execution plan is the thing that says they do not. Reading the plan was the whole difference between reporting an improvement and finding out there was not one.
- A hint that is ignored looks exactly like a hint that worked. Forcing the index on Q1 produced the full scan again, silently, because no legal plan using that index existed. Only the plan output shows the difference.
- Fewer blocks read is not automatically less time. The covering index cut Q1's logical reads by 43 percent and made it slower, because the table already fit in the buffer cache and there was no physical I/O for the saving to save. Collecting both numbers is what caught it.
- PURGE is what turns a dropped table into a real recovery exercise. Without it the recycle bin answers the question and the backup is never tested.
- A recovery is not finished when RMAN says so. The fingerprint, the row counts, the constraints and the untouched departments table are what show that the right point in time was reached and that nothing else moved to get there.